PENALTY LEARNING FOR OPTIMAL PARTITIONING VIA INTERVAL REGRESSION

By

TUNG L NGUYEN

A Dissertation

Submitted in Partial Fulfillment

of the Requirements for the Degree of

Doctor of Philosophy

in Informatics and Computing

Northern Arizona University

May 2026

Approved:

Jarrett Jay Barber, Ph.D., Chair

Toby Dylan Hocking, Ph.D., Co-chair (advisor)

Lan Zhang, Ph.D.

Alex Groce, Ph.D.

ABSTRACT

PENALTY LEARNING FOR OPTIMAL PARTITIONING VIA INTERVAL REGRESSION

TUNG L NGUYEN

Changepoint detection identifies points in a sequence where the underlying data distribution changes, and it is widely used in fields such as finance, genomics, and healthcare. Dynamic programming algorithms, particularly those in the Optimal Partitioning (OP) family, are commonly used to detect changepoints efficiently. These methods rely on a penalty parameter to control the number of detected changepoints, but choosing an appropriate penalty is challenging: too low a penalty can lead to over-segmentation, while too high a penalty can miss important changes. Traditionally, penalty selection has been based on manually computed features of the sequence and simple predictive models, such as linear or tree-based regression. However, these approaches often fail to capture the complex, non-linear relationship between sequence characteristics and the optimal penalty. Furthermore, the optimal penalty is not a single value but lies within a range, which naturally motivates modeling the problem as interval regression. This thesis explores interval regression as a framework for learning penalty values in changepoint detection. I review existing interval regression methods and compare their performance on both synthetic and real-world datasets. Building on this, I propose learning-based approaches for predicting penalties directly from data, including multilayer perceptrons (MLPs) for continuous prediction and recurrent neural networks (RNNs) for sequences with temporal dependencies. These models reduce the need for manual feature engineering and better capture non-linear patterns in the data. Experiments on benchmark genomic datasets show that interval regression–based penalty learning improves changepoint detection performance, achieving higher accuracy and F1 scores than traditional approaches. The results also indicate that no single model works best in all cases, highlighting the importance of selecting a model suited to the characteristics of the data. Overall, this work demonstrates that interval regression provides a natural and effective approach to learning penalties for changepoint detection.

Contents

ABSTRACT ii

TABLE OF CONTENTS iii

LIST OF TABLES iv

LIST OF FIGURES v

PREFACE vi

1 Literature Review: Interval Regression 1
- 1.1 AFT Models . . . 3
  - 1.1.1 Original AFT Model . . . 3
  - 1.1.2 AFT Model in XGBoost . . . 7
- 1.2 Supervised Machine Learning Models . . . 8
  - 1.2.1 Linear . . . 9
  - 1.2.2 Tree . . . 11

2 Literature Review: Optimal Partitioning 12
- 2.1 Changpoint Detection Algorithms . . . 12
- 2.2 Relationship between OPART and Interval Regression . . . 15

3 Literature Gaps and Proposed Plans 16
- 3.1 Inteval Regression . . . 16
- 3.2 Changepoint Detection . . . 17

4 My Contributions 18
- 4.1 Optimal Partitioning Penalty Learning: MLP . . . 18
  - 4.1.1 Contribution . . . 21
  - 4.1.2 Experiments and Results . . . 22
  - 4.1.3 Discussion and Conclusion . . . 25
- 4.2 Optimal Partitioning Penalty Learning: Recurrent . . . 29
  - 4.2.1 Contribution . . . 31
  - 4.2.2 Experiments and Results . . . 33
  - 4.2.3 Discussion and Conclusion . . . 33
- 4.3 Interval Regression: Review . . . 38
  - 4.3.1 Contribution . . . 38
  - 4.3.2 Experiments and Results . . . 39
  - 4.3.3 Discussions and Conclusions . . . 39

List of Tables

1 Comparison of Standard Regression versus Interval Regression setting. 1

2 List of Regression models with compatible censored target intervals. 3

3 List of the regression models in this with corresponding source. 7

4 List of employed penalty prediction models 21

5 Datasets used in the study 22

6 The comparison of the performance and consistency of 7 interval regression models 41

List of Figures

1 Example of point regression versus interval regression 4

2 Example of converting Interval Regression into Standard Regression 5

3 Visualization of the AFT XGBoost Loss (negative log-likelihood) for 3 distributions 8

4 Visualization of the Loss Functions: Error values relative to predictions and targets 9

5 Changepoint detection using Deep Learning 12

6 The challenge of using OPART is predicting the value for penalty 13

7 Different penalty result in varied sets of changepoints 14

8 The process involves a labeled sequences train set and a unlabeled sequences test set 15

9 Comprehensive Diagram: Review and Contributions 18

10 Comparative Model Diagram - Previous Methods versus Our Proposal 19

11 Graphs showing the relationship between four features and target intervals 20

12 Median test accuracy and interquartile across 6 folds for each method 23

13 Median test F1 score and interquartile across 6 folds for each method. 24

14 Average validation accuracies for different MLP configurations 26

15 Example: features vs. targets from the dataset detailed 32

16 The accuracies for each fold are presented for datasets without any pooling 33

17 The fold accuracies on some ChIP-seq datasets 35

18 Log of the loss validation across different numbers of extracted hidden layers . . . 36

19 The mean and standard deviation of the test squared hinge errors in simulated datasets 39

PREFACE

This thesis is the result of several years of study and research on changepoint detection. The work presented here reflects my interest in combining statistical methods with machine learning to address practical challenges in data analysis. I would like to express my sincere gratitude to my primary supervisor Toby Dylan Hocking, and the research community for their guidance, support, and constructive feedback throughout this journey. While this dissertation focuses on technical contributions, it is also a personal exploration of the process of learning, experimentation, and discovery that comes with pursuing research.

Tung Lam Nguyen
nguyenlamtung10@gmail.com

Dissertation

NAU NORTHERN ARIZONA UNIVERSITY
School of Informatics, Computing, and Cyber Systems

1 Literature Review: Interval Regression

Definition In traditional supervised regression tasks, the objective is to develop a model that accurately predicts a continuous real-valued outcome based on a set of input features. This classical problem has been well-studied, resulting in a wide array of algorithms. However, a related but less explored problem is *interval regression*, where each instance is associated with an interval $y = [y_l, y_u]$ – lower and upper – that represents a range of acceptable predictions, rather than a single target value. Unlike conventional regression, where the goal is to predict a specific point, interval regression acknowledges the uncertainty or variability in the target, which is common in various real-world applications, such as Medical Research [Fazil Baksh et al., 2006, Rigaill et al., 2013a, Hawkins, 2004], Economic Forecasting [Abbara and Zevallos, 2018], Environmental Studies [Ellison et al., 2014], Engineering [Levitin et al., 2011]. Refer to Figure 1 for a visualization of the interval regression problem setting.

The outcome data may be uncensored ($-\infty < y_l = y_u < \infty$), right-censored ($-\infty < y_l < y_u = \infty$), left-censored ($-\infty = y_l < y_u < \infty$), or interval-censored ($-\infty < y_l < y_u < \infty$) depending on the available information.

Literature Review Interval-valued outputs commonly occur in survival analysis area. In some studies, Interval Regression models are trained by converting the problem into Standard Regression: instead of using intervals directly as targets, the target interval is represented by discrete points, which are then treated as Standard Regression targets. Using the same notations in Table 1

	training dataset			
	features	target	prediction	this study
Standard Regression	$\mathbf{x} \in \mathbf{R}^m$	$y \in \mathbf{R}$	$\hat{y} \in \mathbf{R}$	✗
Interval Regression	$\mathbf{x} \in \mathbf{R}^m$	$(y_l, y_u) \in \mathbf{R}^2$	$\hat{y} \in \mathbf{R^2}$	✗
	$(\mathbf{x}_l, \mathbf{x}_u) \in \mathbf{R}^{m\times 2}$			✗
	$\mathbf{x} \in \mathbf{R}^m$	$(y_l, y_u) \in \mathbf{R}^2$	$\hat{y} \in \mathbf{R}$	✓

Table 1: Comparison of Standard Regression versus Interval Regression setting.

for $\mathbf{x}$ as the features and (y_l, y_u) as the target interval, Drouin et al. [2017] transform the instance $\big(\mathbf{x}, (y_l, y_u)\big)$ into two points, $(\mathbf{x}, y_l)$ and $(\mathbf{x}, y_u)$, called Interval-CART model – Approach 1 in Figure 2, whereas Cheng et al. [2023] convert it into its midpoint, $(\mathbf{x}, \frac{y_l+y_u}{2})$ – Approach 2 in Figure 2. While these approaches seem straightforward and reasonable, it overlooks the interval information by representing intervals by finite values and fails to handle left-censored or right-censored intervals, resulting in wasted data. Consequently, the performance of these models is worse compared to the models specifically designed for interval targets, see the results in [Drouin et al., 2017, Cheng et al., 2023] and the example in Figure 2.

Linear models for censored outcomes, known as *accelerated failure time* (AFT) models, have been thoroughly explored [Wei, 1992b]. More recently, variants with L1 regularization have emerged to handle high-dimensional datasets [Cai et al., 2009, Huang et al., 2006b]. Nonlinear approaches to censored data have also been investigated, including Support Vector Machines [Pölsterl et al., 2016] and decision tree-based models such as Random Survival Trees [Ishwaran et al., 2008], Cox-Boosting [Binder and Schumacher, 2008], Bagging Survival Trees [Hothorn et al., 2004], Scikit-Survival [Pölsterl, 2020] and Cox-PH in XGBoost [Lundberg et al., 2018]. However, most of these models are limited to right-censored data. Recently, the AFT model in XGBoost [Barnwal et al., 2022b] has emerged as a tree-based model that claims to support all types of label censoring. However, this AFT model does not inherently support negative outcomes, so they addressed this by applying an exponential transformation to the output, ensuring non-negative interval-censored labels. In addition to AFT models, interval regression can also be approached through supervised machine learning techniques that use gradient descent optimizers to minimize a specific loss function (different from negative log-likelihood). This includes L1-regularized linear models with squared hinge loss [Rigaill et al., 2013a] and non-linear models such as maximal margin interval trees (MMIT) [Drouin et al., 2017]. The latter generalizes the widely known Classification and Regression Tree (CART) algorithm [Breiman, 2017] to accommodate censored outputs.

This following sub-sections provides a technical overview of the interval regression method, consisting of two main subsections: AFT models (a statistical approach) and supervised machine

model name	left-censored	right-censored	interval-censored	scalar output	base learner	citation
standard linear	✗	✗	✗	✓	Linear	[Legendre, 1805]
standard knn	✗	✗	✗	✓	kNN	[Fix and Hodges, 1985]
standard tree	✗	✗	✗	✓	Tree	[Breiman et al., 1984]
standard forest	✗	✗	✗	✓	Ensemble trees	[Breiman, 2001]
standard mlp	✗	✗	✗	✓	MLP	[Hornik et al., 1989b]
interval-linear	✓	✓	✓	✗	Linear	[Wang et al., 2015]
quadratic	✓	✓	✓	✗	Linear	[Tanaka and Lee, 1998]
svm	✓	✓	✓	✗	Linear	[Jeng et al., 2003]
aft	✗	✓	✗	✓	Linear	[Wei, 1992a]
reg-aft	✗	✓	✗	✓	Linear	[Huang et al., 2006a]
middlemost	✗	✗	✓	✓	Linear, MLP	[Cheng et al., 2023]
interval-CART	✗	✗	✓	✓	Tree	[Drouin et al., 2017]
interval-knn	✓	✓	✓	✗	kNN	[Hamed et al., 2012]
linear	✓	✓	✓	✓	Linear	[Rigaill et al., 2013a]
mmit	✓	✓	✓	✓	Tree	[Drouin et al., 2017]
aft_xgb	✓	✓	✓	✓	Ensemble trees	[Barnwal et al., 2022a]
constant	✓	✓	✓	✓	Constant	baseline
knn	✓	✓	✓	✓	kNN	proposed
mlp	✓	✓	✓	✓	MLP	proposed
mmif	✓	✓	✓	✓	Ensemble trees	proposed

Table 2: List of Regression models with compatible censored target intervals.

learning models. In essence, AFT models assume the data follows a specific distribution and aim to maximize the corresponding likelihood, while supervised machine learning models focus on minimizing their respective loss functions.

1.1 AFT Models

1.1.1 Original AFT Model

The Accelerated Failure Time (AFT) model is a parametric survival model that directly models the time to an event. The model assumes that covariates act multiplicatively on the survival time, effectively accelerating or decelerating the event. The basic AFT model can be expressed as:

$$\log(T_i) = \mathbf{x}_i^\top \boldsymbol{\beta} + \epsilon_i$$

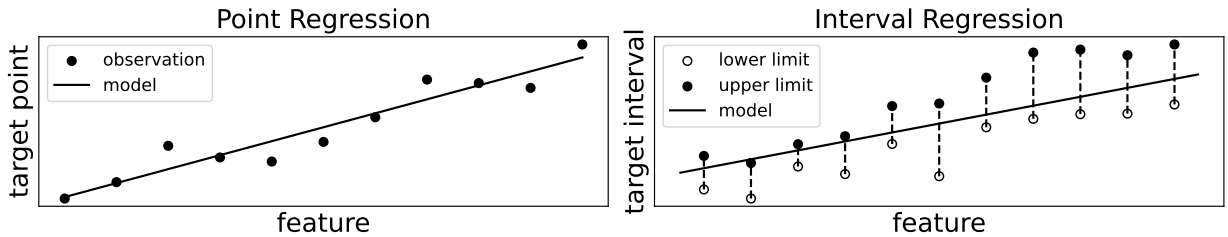

Figure 1: Example of point regression versus interval regression

where:

- T_i is the survival time (non-negative) for the i-th individual,
- $\mathbf{x}_i$ is the vector of covariates for the i-th individual,
- $\boldsymbol{\beta}$ is the vector of regression coefficients,
- ϵ_i is a random error term following a specified distribution (e.g., normal, log-normal, Weibull).

Censored Data and Likelihood Function In survival analysis, the data may be *right-censored*, meaning the event of interest has not occurred by the end of the study. The likelihood function must account for both observed and censored data.

Let δ_i be an indicator variable, where:

$$\delta_i = \begin{cases} 1 & \text{if the event is observed,} \\ 0 & \text{if the observation is censored.} \end{cases}$$

The likelihood function for the AFT model with censored data is:

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} \left[f(T_i \mid \mathbf{x}_i, \boldsymbol{\beta})\right]^{\delta_i} \left[S(T_i \mid \mathbf{x}_i, \boldsymbol{\beta})\right]^{1-\delta_i}$$

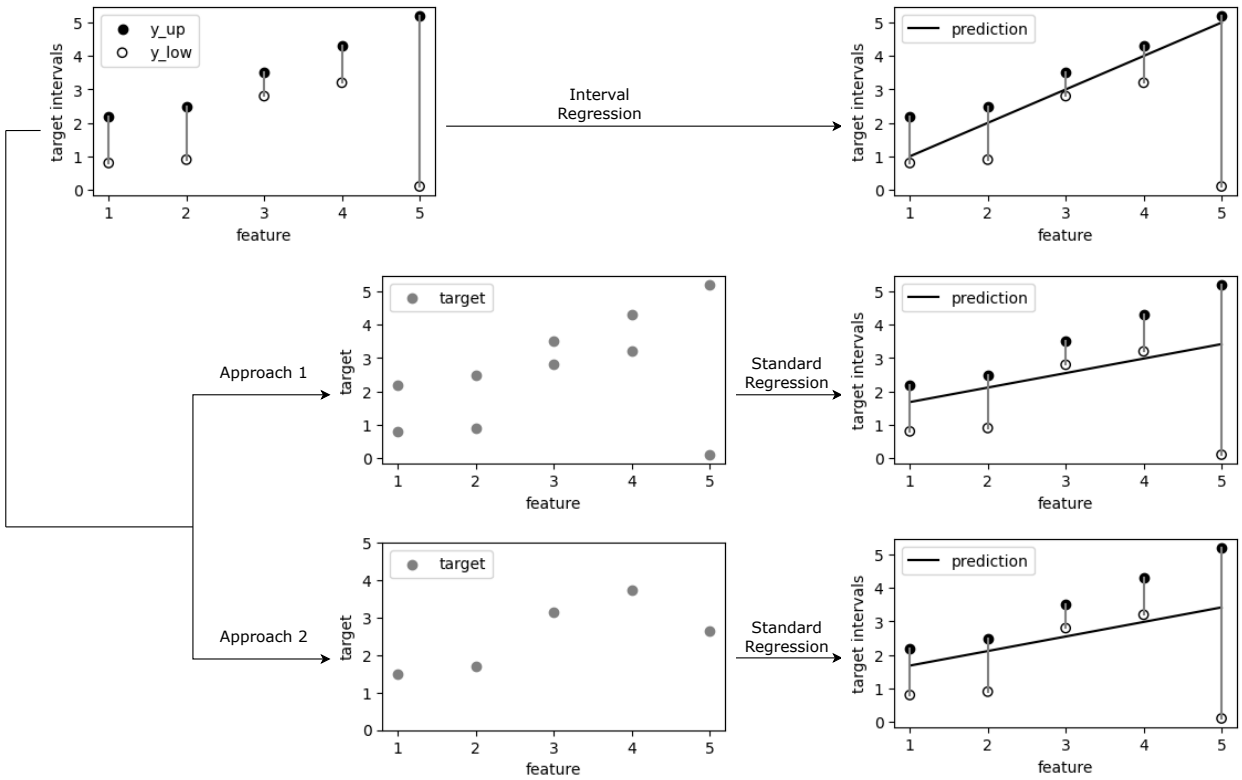

Figure 2: Example of converting Interval Regression into Standard Regression

where $f(T_i \mid \mathbf{x}_i, \boldsymbol{\beta})$ is the probability density function, and $S(T_i \mid \mathbf{x}_i, \boldsymbol{\beta}) = P(T_i > t)$ is the survival function. The survival function represents the probability that the event has not occurred by a certain time t, i.e.,

$$S(T_i \mid \mathbf{x}_i, \boldsymbol{\beta}) = 1 - F(T_i \mid \mathbf{x}_i, \boldsymbol{\beta}),$$

where $F(T_i \mid \mathbf{x}_i, \boldsymbol{\beta})$ is the cumulative distribution function (CDF) of the event time T_i conditional on the covariates $\mathbf{x}_i$ and model parameters $\boldsymbol{\beta}$.

Taking the logarithm of the likelihood function yields the log-likelihood:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left[\delta_i \log f(T_i \mid \mathbf{x}_i, \boldsymbol{\beta}) + (1 - \delta_i) \log S(T_i \mid \mathbf{x}_i, \boldsymbol{\beta})\right]$$

Model Fitting Procedure To estimate the parameters $\boldsymbol{\beta}$, we maximize the log-likelihood function. The general steps for training the AFT model are:

1. Specify the distribution for the error term ϵ_i (e.g., normal, log-normal, Weibull).
2. Define the likelihood function based on the observed and censored data.
3. Maximize the log-likelihood function to estimate $\boldsymbol{\beta}$.
4. Use the estimated parameters to make predictions.

Example: Survival Analysis with Normal Distribution Let us consider the case where the survival times follow a normal distribution, i.e., $\epsilon_i \sim N(0, \sigma^2)$. The model is given by:

$$\log(T_i) = \beta_0 + \beta_1 x_{i1} + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2)$$

The corresponding log-likelihood function for this model is:

$$\ell(\boldsymbol{\beta}, \sigma) = \sum_{i=1}^{n} \left[\delta_i \log \left(\frac{1}{\sigma\sqrt{2\pi}} \exp \left(-\frac{(\log T_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2}{2\sigma^2} \right) \right) + (1 - \delta_i) \log S(T_i \mid \mathbf{x}_i, \boldsymbol{\beta}, \sigma) \right]$$

where $S(T_i \mid \mathbf{x}_i, \boldsymbol{\beta}, \sigma)$ is the survival function for the normal distribution:

$$S(T_i \mid \mathbf{x}_i, \boldsymbol{\beta}, \sigma) = 1 - \Phi \left(\frac{\log T_i - \mathbf{x}_i^\top \boldsymbol{\beta}}{\sigma} \right)$$

and $\Phi(\cdot)$ is the cumulative distribution function (CDF) of the standard normal distribution.

The parameters β_0, β_1, and σ are estimated by maximizing the log-likelihood function. Predictions for new individuals can be made by:

$$\hat{T}_{\text{new}} = \exp(\mathbf{x}_{\text{new}}^\top \hat{\boldsymbol{\beta}} + \hat{\epsilon})$$

model	source code	language	citation
linear	[Hocking, 2024]	R	[Rigaill et al., 2013a]
mmit	[Drouin, 2017]	C++	[Drouin et al., 2017]
aft_xgb	[Chen and Guestrin, 2016]	Python	[Barnwal et al., 2022a]
constant	paper GitHub repo	C++	additional baseline
knn	paper GitHub repo	Python	proposed
mlp	paper GitHub repo	Python	proposed
mmif	paper GitHub repo	C++	proposed

Table 3: List of the regression models in this with corresponding source.

where $\hat{\epsilon}$ is drawn from $N(0, \hat{\sigma}^2)$.

1.1.2 AFT Model in XGBoost

The AFT model in XGBoost, introduced by Barnwal et al. [2022a], is an ensemble tree-based model designed for survival analysis. The model makes predictions as follows:

$$y_p = \mathbf{T}(\mathbf{x}),$$

where $\mathbf{T}$ represents the ensemble of trees in XGBoost. Although it belongs to the AFT model family, this model can handle all non-negative target intervals. To extend its applicability to real-valued target intervals $(y_l, y_u) \in \mathbf{R}^2$, Barnwal et al. [2022a] apply an exponential transformation to the original interval:

$$(y_l, y_u) \longrightarrow (\exp y_l, \exp y_u) \in \mathbf{R}_+^2$$

After making a prediction y_p in the exponential space, the result is mapped back to the original scale using the logarithm transformation:

$$\hat{y} = \log y_p = \log \mathbf{T}(\mathbf{x})$$

Training this model is based on minimizing the negative log-likelihood function (loss function),

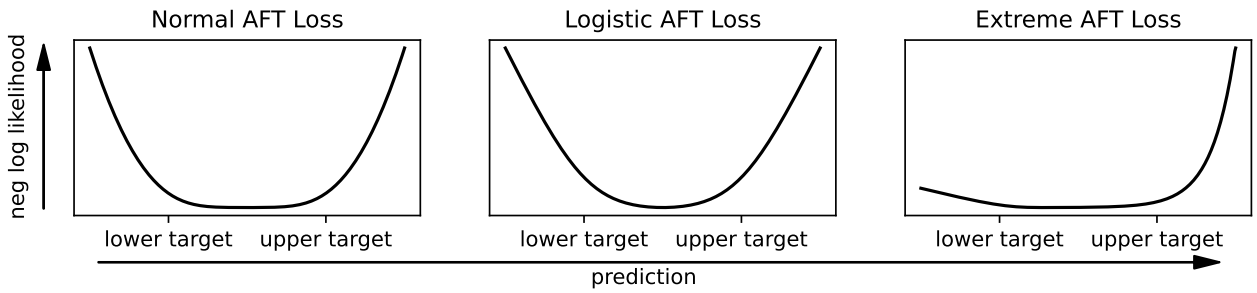

Figure 3: Visualization of the AFT XGBoost Loss (negative log-likelihood) for 3 distributions

which incorporates all training survival instances:

$$l_{AFT}\Big(\hat{y}, (y_l, y_u)\Big) = -\log\Big[F_Z\big(\frac{y_u - \hat{y}}{\sigma}\big) - F_Z\big(\frac{y_l - \hat{y}}{\sigma}\big)\Big] \tag{1}$$

where $\sigma > 0$ is the scale parameter. The cumulative distribution function $F_Z(z)$ depends on the chosen distribution (normal, logistic, or extreme):

- normal: $F_Z(z) = \frac{1}{2}\big(1 + \text{erf}(\frac{z}{\sqrt{2}})\big)$
- logistic: $F_Z(z) = \frac{e^z}{1+e^z}$
- extreme: $F_Z(z) = 1 - e^{-\exp z}$

Figure 3 visualizes the loss function for each distributions, which can be compared to Figure 4 to observe their similarities. The model is trained using the gradient boosting algorithm [Friedman, 2001] to minimize the loss value (1) over the training dataset with N instances $\{\mathbf{x}^i, (y_l^i, y_u^i)\}_{i=1}^N$ given the distribution and the scale parameter σ:

$$\sum_{i=1}^{N} l_{AFT}\Big(\hat{y}^i, (y_l^i, y_u^i)\Big) = \sum_{i=1}^{N} -\log\Big[F_Z\big(\frac{y_u^i - \log \mathbf{T}(\mathbf{x}^i)}{\sigma}\big) - F_Z\big(\frac{y_l^i - \log \mathbf{T}(\mathbf{x}^i)}{\sigma}\big)\Big]$$

1.2 Supervised Machine Learning Models

Before delving into the methods, it is essential to clarify the loss function that these models utilized in interval regression. While the mean squared error (MSE) or mean absolute error (MAE) is com-

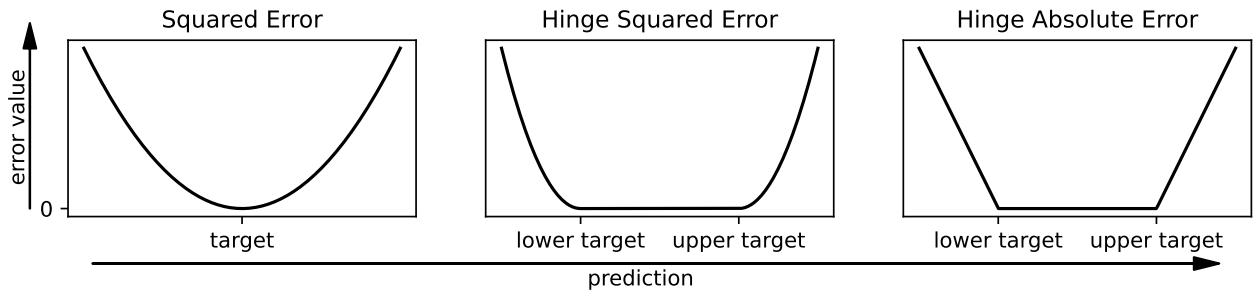

Figure 4: Visualization of the Loss Functions: Error values relative to predictions and targets

monly used for traditional regression tasks, interval regression employs a related criterion known as hinge error. The hinge loss function, leveraging the ReLU activation function, is defined as:

$$l(\hat{y}, y) = l(\hat{y}, [y_l, y_u]) = \big(\text{ReLU}(y_l - \hat{y} + \epsilon)\big)^p + \big(\text{ReLU}(\hat{y} - y_u + \epsilon)\big)^p \tag{2}$$

where $\epsilon \geq 0$ is the margin (or hinge), $\hat{y}$ denotes the predicted value and $y = [y_l, y_u]$ represents the target interval with $y_l \leq y_u$. p is either 1 or 2 ($p = 1$ is equivalent to MAE, $p = 2$ is equivalent to MSE). This formulation ensures that the loss is minimized when the predicted value $\hat{y}$ falls within the target interval $[y_l, y_u]$. Unlike MSE or MAE, which achieves zero loss at a single target value, the hinge error function yields zero loss within a specified target interval, as illustrated in Figure 4.

From this point forward in the study, the following notation will be used: $\mathbf{x}$ represents the feature vector, $[y_l, y_u]$ denotes the target interval, and $\hat{y}$ stands for the predicted output.

1.2.1 Linear

This linear model was introduced in the first time by Rigaill et al. [2013a], and its extended theoretical foundations were later studied by Cheng et al. [2023]. The prediction formula is given:

$$\hat{y} = \mathbf{x} \cdot \beta + \beta_0 \tag{3}$$

where $\beta \in \mathbf{R}^m$ and $\beta_0 \in \mathbf{R}$ are the parameters to be learned. To estimate these parameters, the authors extended the Standard Regression loss function—specifically, the Squared Error or Mean

Squared Error (MSE)—into a generalized form called the *Hinge Squared Error*. The error value between prediction $\hat{y}$ and target interval (y_l, y_u) is defined as follows:

$$l\Big(\hat{y}, (y_l, y_u)\Big) = \begin{cases} (y_l - \hat{y} + \epsilon)^2, & \text{if } \hat{y} < y_l + \epsilon \\ 0, & \text{if } y_l + \epsilon \leq \hat{y} \leq y_u - \epsilon \\ (\hat{y} - y_u + \epsilon)^2, & \text{if } \hat{y} > y_u - \epsilon \end{cases}$$

where $\epsilon \geq 0$ is the margin length ($\epsilon = 0$ by default). The loss function becomes MSE when $y_l = y_u$ and $\epsilon = 0$, or in other word, target interval becomes target point.

Cheng et al. [2023] proposed a model called *Regression with Interval Targets* (RIT), which is essentially the same as model (3), but uses *Hinge Absolute Error* instead:

$$l\Big(\hat{y}, (y_l, y_u)\Big) = \begin{cases} |y_l - \hat{y} + \epsilon|, & \text{if } \hat{y} < y_l + \epsilon \\ 0, & \text{if } y_l + \epsilon \leq \hat{y} \leq y_u - \epsilon \\ |\hat{y} - y_u + \epsilon|, & \text{if } \hat{y} > y_u - \epsilon \end{cases}$$

The general Interval Regression loss function can be written in this short form:

$$l\Big(\hat{y}, (y_l, y_u)\Big) = \Big(\text{ReLU}(y_l - \hat{y} + \epsilon)\Big)^p + \Big(\text{ReLU}(\hat{y} - y_u + \epsilon)\Big)^p \tag{4}$$

where $p \in \{1, 2\}$. Refer to Figure 4 for the visual comparison between hinge errors (Interval Regression Loss) and squared error (Standard Regression Loss).

To estimate the parameters β and β_0 in (3), apply a gradient descent algorithm with respect to β and β_0 to minimize the Hinge Error (4) over the training dataset having N instances $\{\mathbf{x}^i, (y_l^i, y_u^i)\}_{i=1}^N$:

$$\sum_{i=1}^{N} l\Big(\hat{y}^i, (y_l^i, y_u^i)\Big) = \sum_{i=1}^{N} l\Big(\mathbf{x^i} \cdot \beta + \beta_0, (y_l^i, y_u^i)\Big)$$

Rigaill et al. [2013a] employed the Fast Iterative Shrinkage-Thresholding Algorithm or FISTA

[Beck and Teboulle, 2009], while Cheng et al. [2023] employed stochastic gradient descent (SGD).

1.2.2 Tree

MMIT introduced by Drouin et al. [2017] is applicable for λ prediction as an interval regression model. The MMIT model is:

$$\log \hat{\lambda} = \mathcal{T}(\mathbf{x})$$

The tree $\mathcal{T}$ architecture follows the same structure as CART (Classification And Regression Tree) [Breiman et al., 1984]. The only difference lies in the regression value for each leaf which is a set of Q instances $\{\mathbf{x}^i, [\log \lambda_l^i, \log \lambda_u^i]\}_{i=1}^Q$: instead of taking the mean of all targets like CART does, it chooses a constant value c that minimizes the mean Hinge Error (2) between this constant and the target intervals:

$$\underset{c}{\operatorname{argmin}} \frac{1}{Q} \sum_{i=1}^{Q} l\big(c, [\log \lambda_l^i, \log \lambda_u^i]\big), \quad c \in \mathbf{R}$$

A heuristic algorithm is used to speed up the estimation of the mean value for each leaf by considering a finite set of candidates, rather than solving for the optimal c:

$$\underset{c}{\operatorname{argmin}} \frac{1}{Q} \sum_{i=1}^{Q} l\big(c, [\log \lambda_l^i, \log \lambda_u^i]\big),$$
$$\text{where } c \in \{\log \lambda_l^i, \log \lambda_u^i, \frac{\log \lambda_l^i + \log \lambda_u^i}{2} | \log \lambda_l^i > -\infty, \log \lambda_u^i < \infty\}_{i=1}^Q$$

The training procedure for determining the optimal tree architecture follows the same operations as in CART (binary optimization) to minimize the Hinge Error (2) over the train dataset having S instances $\{\mathbf{x}^i, [\log \lambda_l^i, \log \lambda_u^i]\}_{i=1}^S$:

$$\sum_{i=1}^{S} l\Big(\log \hat{\lambda}^i, [\log \lambda_l^i, \log \lambda_u^i]\Big) = \sum_{i=1}^{S} l\Big(\mathcal{T}(\mathbf{x}^i), [\log \lambda_l^i, \log \lambda_u^i]\Big)$$

The optimal tree architecture (maximum depth, minimum split), loss type which is p (either 1 or 2) in Hinge Error (2), and margin which is ϵ in Hinge Error (2), is selected through cross-validation.

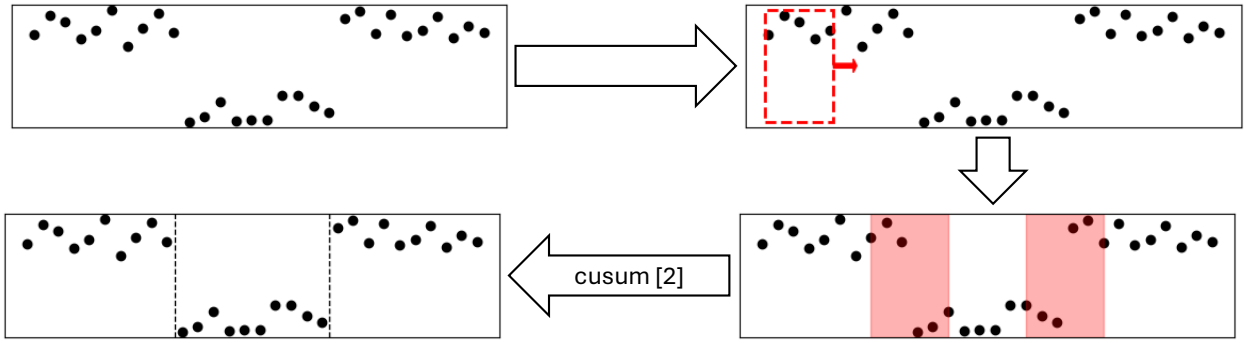

Figure 5: Changepoint detection using Deep Learning

2 Literature Review: Optimal Partitioning

2.1 Changpoint Detection Algorithms

Changepoint detection serves as a vital tool in numerous real-life applications by pinpointing significant transitions or sudden changes in data patterns, ranging from detecting market trends in finance [Lattanzi and Leonelli, 2021], monitoring disease outbreaks in healthcare [Muggeo and Adelfio, 2010], enhancing network security [Tartakovsky et al., 2013], monitoring environmental changes [Reeves et al., 2007] and more.

Changepoint detection methods can be divided into two categories: Deep Learning (DL) and Dynamic Programming (DP). DL-based methods, which inspired from a statistical method Cumulative Sum (CUSUM) [Hinkley, 1971, James et al., 1987], including recent works [Li et al., 2024, Ermshaus et al., 2023] employ multilayer perceptrons (MLP) or k-nearest neighbors (KNN) as classification sliding windows to identify changepoints. Take a look at the Figure 5 for a clear visualization of the operation. However, these methods rely on local neighbors rather than the entire sequence and are overly complex for univariate data. In contrast, DP is well-suited for this context.

To determine the optimal partition of a sequence given a fixed number of segments (or number of changepoints), several efficient algorithms can be employed to achieve precise results [Auger and Lawrence, 1989a, Bai and Perron, 2003a, Killick et al., 2012a]. However, in many instances, the number of segments is not predetermined and must instead be inferred from the data. To ad-

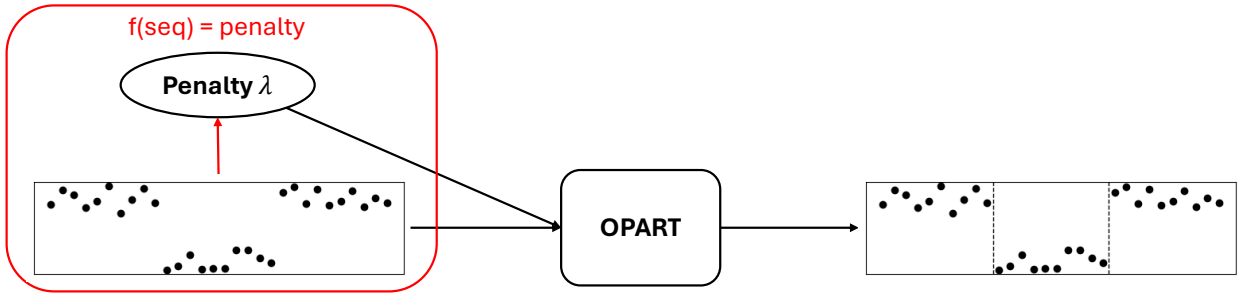

Figure 6: The challenge of using OPART is predicting the value for penalty

dress this challenge, several dynamic programming algorithms have been successfully applied to changepoint detection, including Optimal Partitioning (OPART) [Jackson et al., 2005a] with its variations such as Pruned Exact Linear Time (PELT) [Killick et al., 2012a] or Functional Pruning Optimal Partitioning (FPOP) [Maidstone et al., 2016a]. PELT uses a theorem to prune certain candidate changepoints that cannot be part of the optimal partition. FPOP prunes at least as many candidate changepoints as PELT and can be more efficient, particularly for large datasets. Despite these differences, OPART, PELT, and FPOP all produce the same partition. The Labeled Optimal Partitioning (LOPART) [Hocking and Srivastava, 2023a] extends OPART's capabilities by leveraging predefined changepoint labels, indicating the expected number of changepoints within specific location ranges. When no such labels are defined, LOPART behaves identically to OPART. Without prior knowledge of the number of changepoints in the partition and aiming to limit their occurrence, these algorithms employ a penalty parameter to discourage the presence of changepoints, denoted as λ, in the following optimization problem

OPART problem Find partition vector $\mathbf{m} \in \mathbb{R}^N$ that minimizes the following cost function for a given sequence $\mathbf{d} \in \mathbb{R}^N$ and penalty parameter $\lambda \geq 0$:

$$\mathbf{m} = \underset{\mathbf{m}}{\operatorname{argmin}}\, C_\lambda(\mathbf{d}, \mathbf{m}) \tag{5}$$

$$\text{where} \quad C_\lambda(\mathbf{d}, \mathbf{m}) = \sum_{i=1}^{N} l(d_i, m_i) + \lambda I[m_i \neq m_{i+1}] \tag{6}$$

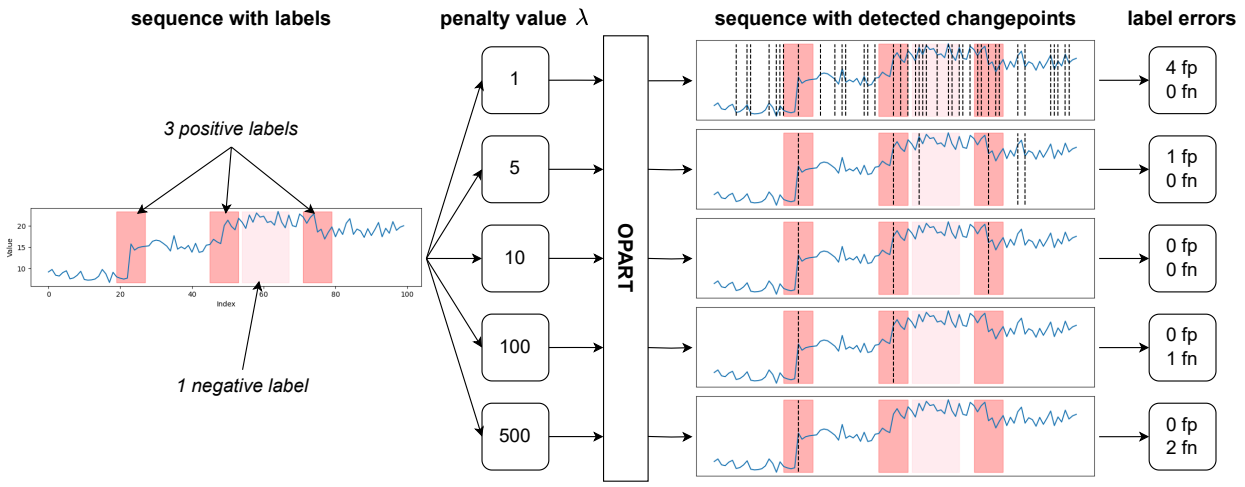

Figure 7: Different penalty result in varied sets of changepoints

where $l(m_i, d_i)$ usually represents the squared error between the sequence value d_i and the mean segment m_i, i.e., $l(d_i, m_i) = (m_i - d_i)^2$. $I[m_i \neq m_{i+1}]$ is an indicator function that equals 1 if there is a changepoint (when $m_i \neq m_{i+1}$) and 0 otherwise. Thus, $k = \sum_{i=1}^{N} I[m_i \neq m_{i+1}]$ represents the total number of changepoints, hence $k + 1$ is the number of segments. For instance, for a sequence $\mathbf{d} = [d_1, d_2, d_3, d_4, d_5, d_6] \in \mathbb{R}^6$, if we have a fixed penalty λ^*, the optimal partition vector of problem 4.2 is $\mathbf{m} = [m_1, m_1, m_1, m_1, m_2, m_2]$, this indicates that the partition $\mathbf{m}$ consists of 1 changepoint and 2 segments: $[d_1, d_2, d_3, d_4]$ and $[d_5, d_6]$.

Role of the penalty parameter λ Within the OPART problem 4.2, the penalty parameter λ holds significant importance in partitioning. Moreover, a higher λ imposes a more substantial penalty for changepoints' presence, consequently yielding smaller sets of changepoints. When $\lambda = 0$, the optimal partition $\mathbf{m}$ in problem 4.2 matches the given sequence $\mathbf{d}$ exactly ($\mathbf{m} \equiv \mathbf{d}$), while as λ approaches $+\infty$, all values in $\mathbf{m}$ is the mean of $\mathbf{d}$. A too high value of λ is undesirable because it results in fewer detected changepoints, potentially missing significant ones. Conversely, a too low value of λ results in an excessive number of detected changepoints, more than necessary. While the existing changepoint detection algorithms has demonstrated effectiveness, the fixed nature of λ prompts inquiry into methods for dynamically adapting this critical parameter.

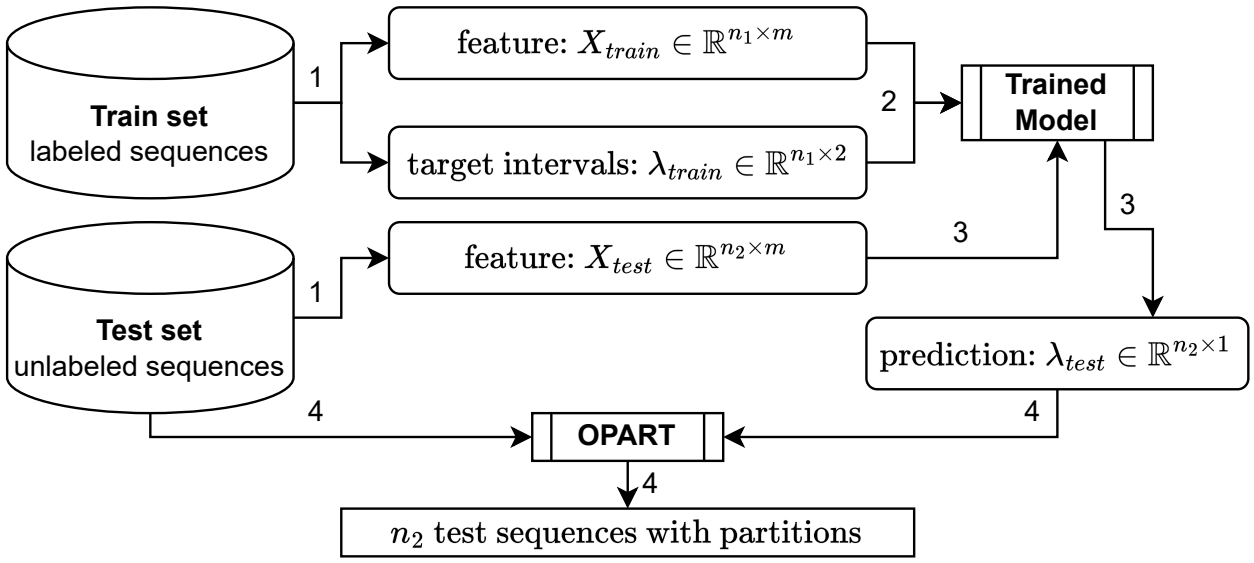

Figure 8: The process involves a labeled sequences train set and a unlabeled sequences test set

2.2 Relationship between OPART and Interval Regression

OPART algorithms take into the sequence $\mathbf{d}$ along with λ as input to generate the set of detected changepoints. Each labeled sequence $\mathbf{d}^j$ has M_j distinct labels $\{(s_i, e_i, c_i)\}_{i=1}^{M_j}$, where c_i denotes the expected number of changepoints between start location s_i and end location e_i. There are two types of label errors: in each label, if the number of detected changepoints greater than c_i, it's a *false positive*; if smaller, it's a *false negative*.

Finding the optimal partition for a given labeled sequence require considering a range of values for λ (rather than just one specific value) that minimizes the number of label errors. Refer to [Rigaill et al., 2013a] for a method to determine the optimal range of λ for each labeled sequence. Consequently, the value of λ to be predicted should fall within an optimal interval, see Figure 1. This type of problem, where the objective is to find values falling within an interval, is referred to as the *interval regression*.

3 Literature Gaps and Proposed Plans

3.1 Inteval Regression

Limitation Let me briefly summarize the previous works and highlight their limitations:

- **Original AFT model and its variants**: Designed for survival analysis, it handles only uncensored and right-censored data, as it does not account for left-censored data.
- **AFT model in XGBoost**: Capable of handling all types of censored data by transforming target intervals to ensure non-negative values. It introduces a loss function for all types of censored outcomes. However, being tree-based, the model lacks smooth decision boundaries.
- **Linear model in supervised machine learning**: Similar to the original AFT model, it has a simple linear architecture that struggles with datasets exhibiting non-linear patterns.
- **MMIT**: A non-linear model that captures complex relationships, but like any tree-based models, it produces non-smooth decision boundaries, leading to a lack of smoothness in predictions.

Proposed Plan My plan involves utilizing more sophisticated and smoother models, such as Generalized Additive Models (GAMs) [Hastie, 2017], random forest, or like multi-layer perceptrons (MLP).

- One option would be an MLP with just one hidden layer and very few neurons, using Rectified Linear Unit activation functions like ReLU or Leaky ReLU (not ELU or SELU or anything that creates non-linear regions). For instance, an MLP with one hidden layer and two neurons, using Leaky ReLU as the activation function, can be about twice as complex as a linear model, as it can create up to four linear regions (see Proposition 3 in [Montufar et al., 2014]). This makes it a more general approach than traditional linear models without being overly complex.

- Alternatively, for a non-linear model that remains relatively simple, I could opt for piecewise linear models, like splines [Friedman, 1991] with linear terms in each region.

3.2 Changepoint Detection

Limitation The OPART algorithm employs a fixed penalty value, λ, which may limit its adaptability to different data structures.

Proposed Plan

- Explore heuristic approaches that divide the data into smaller segments, followed by applying optimal partitioning to each subsequence.
- Consider applying k-means clustering to detect potential segments.
- Applying complex models for penalty prediction.

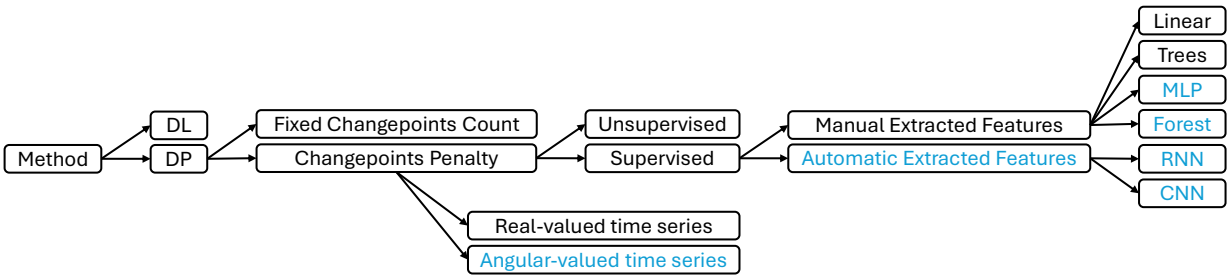

Figure 9: Comprehensive Diagram: Review and Contributions

4 My Contributions

4.1 Optimal Partitioning Penalty Learning: MLP

Changepoint detection (or other words, partitioning and segmentation) is essential in many real-world applications, identifying significant data shifts in finance [Lattanzi and Leonelli, 2021], healthcare [Muggeo and Adelfio, 2010], network security [Tartakovsky et al., 2013], environmental monitoring [Reeves et al., 2007], and more.

Detecting changepoints involves selecting the detection method and tuning hyperparameters to detect the changepoint locations.

Changepoint detection methods review Many real-life time series data are univariate. Several deep learning (DL)-based changepoint detection methods exist [Londschien et al., 2023, Lee et al., 2023, Li et al., 2024, Ermshaus et al., 2023], these approaches generalize classical methods [Hinkley, 1971, James et al., 1987] by applying sliding classification windows built using either multilayer perceptron (MLP) or k-nearest neighbors (KNN). DL-based methods are fairly easy to understand and implement, making them accessible to a wide audience. But these methods require tuning numerous hyperparameters, making them overly complex for univariate data. In contrast, dynamic programming (DP) algorithms provides an ideal solution in this context. *This study selects DP to enhance its changepoint detection performance.*

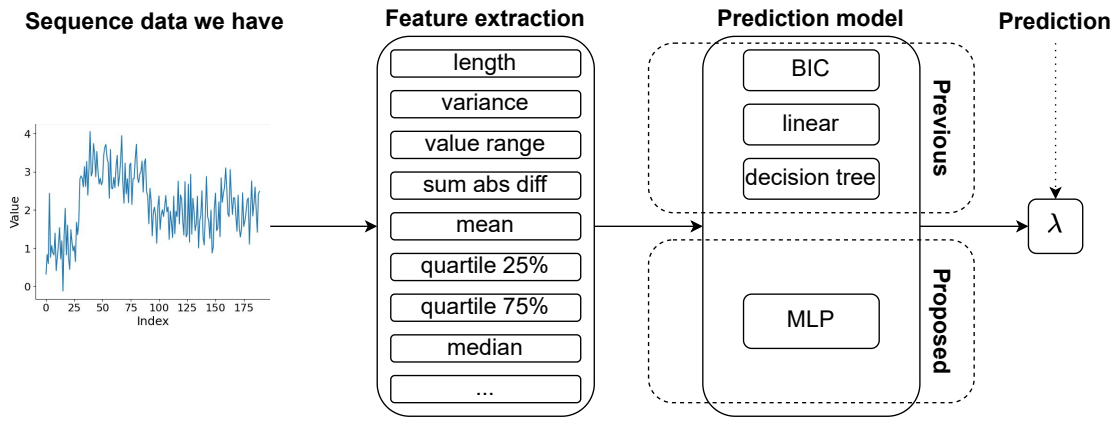

Figure 10: Comparative Model Diagram - Previous Methods versus Our Proposal

Regularization for Changepoints Count in DP Algorithms DP algorithms detects changepoints using only one hyperparameter (contrast to numerous hyperparameters in DL-based methods) to control changepoints count, either (a) a fixed changepoints count or (b) a penalty to penalize changepoint presence. Approach (a), used in [Auger and Lawrence, 1989a, Bai and Perron, 2003a, Killick et al., 2012a], is effective but relies on often unknown changepoints count. Approach (b), with the most popular methods being Optimal Partitioning (OP) [Jackson et al., 2005a] and its variants, including Pruned Exact Linear Time (PELT) [Killick et al., 2012a], Functional Pruning Optimal Partitioning (FPOP) [Maidstone et al., 2016a], and Labeled Optimal Partitioning (LOP) [Hocking and Srivastava, 2023a]. PELT and FPOP improves efficiency by pruning technique whilst producing the same solution as OP. LOP extends OP by incorporating predefined changepoint labels and defaults to OP when labels are absent. *This study selects approach (b) to enhance changepoint detection performance as it offers greater flexibility.*

The role of the OP penalty A higher penalty imposes stricter penalties, reducing the changepoints count (see Figure 7). Given the suitability of OP family algorithms for changepoint detection in univariate sequences and the role of the OP penalty, *this study focuses on developing a model to predict the optimal OP penalty*.

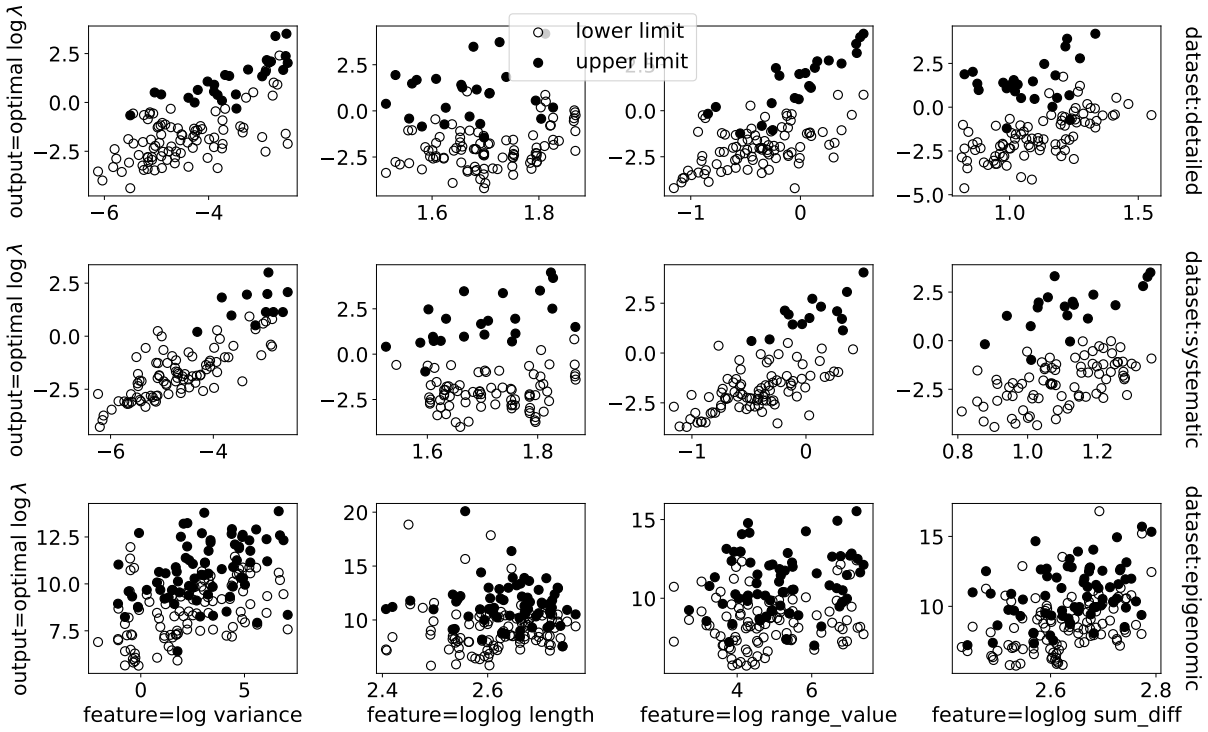

Figure 11: Graphs showing the relationship between four features and target intervals

Related Work review Various methods exist for predicting the OP penalty, including unsupervised [Schwarz, 1978, Akaike, 1974, Lavielle, 2005] and supervised ones [Truong et al., 2017, Rigaill et al., 2013b, Drouin et al., 2017, Barnwal et al., 2022a].

Gap in the Literature and Contributions Existing models are constrained by simplistic architectures, such as linear or tree-based, which motivated us to propose a novel approach. This study proposes MLP with activation function ReLU for OP penalty prediction. This is a generalized version of a linear model and offers continuous nonlinear predictions compared to tree-based models. Experiments conducted on three large genome datasets demonstrate that the proposed model outperforms previous work in accuracy and F1 score. For simplicity, we refer to the OP penalty as λ hereafter.

Model	Features	Model Type	Regularization	Citation
BIC.1	1	unsupervised	None	Schwarz [1978]
linear.1 linear.2 linear.4	1 2 4	linear	None	Hocking and Srivastava [2023a] Rigaill et al. [2013b] Lavielle [2005]
linear.all	118		L1	Rigaill et al. [2013b]
mmit.1 mmit.2 mmit.4 mmit.all	1 2 4 118	tree	max depth min split sample loss type loss margin	Drouin et al. [2017]
aft_xgboost.1 aft_xgboost.2 aft_xgboost.4 aft_xgboost.all	1 2 4 118	ensemble tree	distribution scale learning rate max depth min child weight reg alpha reg lambda	Barnwal et al. [2022a]
mlp.1 mlp.2 mlp.4 mlp.all	1 2 4 118	MLP	hidden layers count neurons per layer early stopping	proposed

Table 4: List of employed penalty prediction models

4.1.1 Contribution

Novelty: MLP for λ prediction A MLP with fully connected architecture is used. All hidden layers have the same number of neurons. ReLU activation function is used in the input and hidden layer(s). This architecture makes the proposed model a more general form of a linear model and produces continuous nonlinear predictions compared to tree-based models like MMIT and AFT in XGBoost.

Additional Contribution: Feature Selection A meaningful feature distinguishes sequences from one another in the context of changepoint detection. Features such as the mean, quartile values (min, Q1, Q2, Q3, max) and their transformations are not selected because these features are noise in the changepoint context. For example, $[0, 0, 0, 1, 1, 1]$ and $[1, 1, 1, 2, 2, 2]$ are the same sequences.

Dataset	Sequences	Pos. Labels	Neg. Labels	Source
Detailed	3730	964	3396	GitHub
Systematic	3418	573	2846	GitHub
Epigenomic	4913	2185	8195	UCI Repo

Table 5: Datasets used in the study

Sequence length and variance alone cannot fully represent a sequence, since some sequences may have the same length and variance but differ significantly in this context, so we introduce two additional sequence-related features: *value range* and *sum of absolute differences*.

- **Value range:** This feature (r in Table 3) is defined as the difference between the maximum and minimum values in the sequence.
- **Sum of absolute differences:** This feature (s in Table 3) is calculated as the sum of the absolute differences between two consecutive sequence points, denoted as $\sum_{i=1}^{N-1} |d_{i+1} - d_i|$, where $\mathbf{d} = [d_1, d_2, \ldots, d_N]$ is the sequence. This feature differentiates sequences with the same length, variance, or value range, such as [0,0,0,1,1,1] and [0,1,0,1,0,1], which are distinct in the context of changepoint detection.

Intuitively, as the value range or the sum of absolute differences increases, the sequence tends to show more fluctuations, indicating a need to increase λ. See Figure 11 to see why these two features emerge as strong candidates.

I employed cross-validation to determine the appropriate MLP architecture, which was time-consuming, for details, see Figure 14. My approach achieved improved accuracy rates in change-point detection when applied to benchmark datasets (Figure 12). I have uploaded my research to arXiv: link. And it's published in Statistics and Computing journal: Spring Nature link.

4.1.2 Experiments and Results

Features Four sets of features are considered to evaluate their quality:

- **1 feature**: Sequence length, as used in Hocking and Srivastava [2023a] and Schwarz [1978].

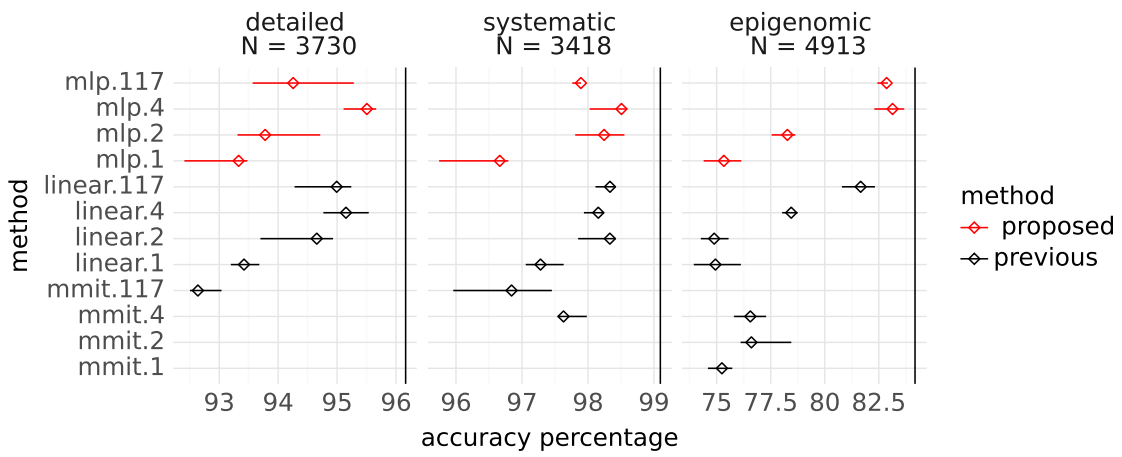

Figure 12: Median test accuracy and interquartile across 6 folds for each method

- **2 features**: Sequence length and variance, following Rigaill et al. [2013b].
- **4 features**: Sequence length, variance, value range, sum of absolute differences.
- **All features**: According to Rigaill et al. [2013b] and Drouin et al. [2017], up to 365 features are generated, with only those without of missing data, infinite values, or zero variance retained.

Models Configuration All models employed 5-fold cross-validation to select the optimal hyperparameters.

- **Linear**: L1 regularization is applied only to the large feature vector, with cross-validation used to determine the optimal L1 parameter value, starting at 0.001 and increasing by 1.2 until no features remain.
- **MMIT**: The hyperparameters considered include a list for `max_depth` values (0, 1, 5, 10, 20, Inf), `margin` (0, 1, 2) (ϵ from equation 2), `loss_type` (`hinge`, `square` – equivalent to $p = 1$ or $p = 2$ in Equation 2) and `min_sample` (0, 1, 2, 4, 8, 16, 20).
- **AFT in XGBoost**: The hyperparameters considered include a list of `learning_rate` values (0.001, 0.01, 0.1, 1.0), `max_depth` (2, 3, 4, 5, 6, 7, 8, 9 10), `min_child_weight`

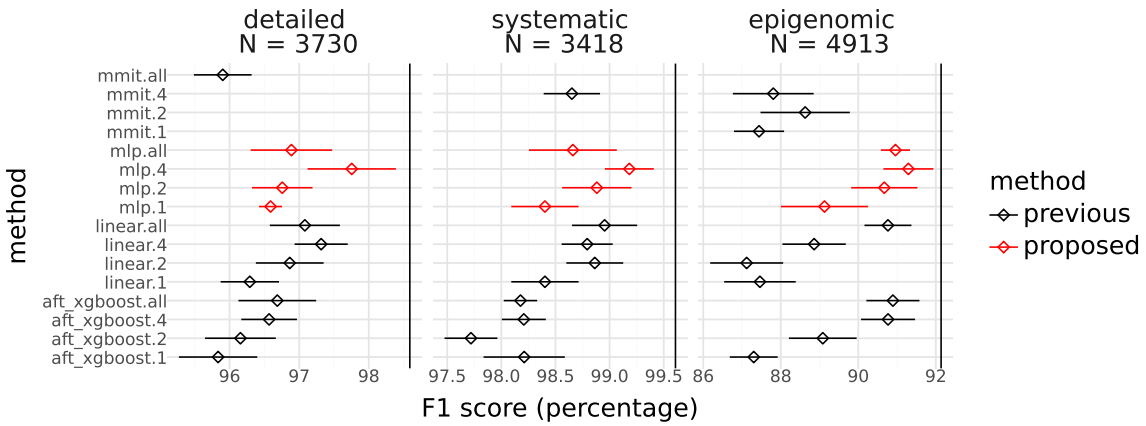

Figure 13: Median test F1 score and interquartile across 6 folds for each method.

(0.001, 0.1, 1.0, 10.0, 100.0), `reg_alpha` (0.001, 0.01, 0.1, 1.0, 10.0, 100.0), `reg_lambda` (0.001, 0.01, 0.1, 1.0, 10.0, 100.0), `aft_loss_distribution_scale` (0.5, 0.8, 1.1, 1.4, 1.7, 2.0). This setup is identical to that described in [Barnwal et al., 2022b].

- **MLP**: The hyperparameters considered include a list of `n_hidden_layer` (1, 2, 3, 4) and `hidden_layer_size` (2, 4, 8, 16, 32, 64, 128, 256, 512). We used the early stopping technique using a `patience` parameter, stopping training if the loss does not decrease after the specified patience iterations; in our study, we set the maximum iterations to 12,000 and the patience to 20. The model is trained using the Adam optimizer (with default PyTorch hyperparameters) to minimize the Squared Hinge Loss (as defined in Function 2 with $p = 2$ and $\epsilon = 1$).

Results This study uses 3 large datasets, as detailed in Table 4.1.1. Each dataset is split into 6 similar size folds. In each iteration, 1 fold is designated as the test set, and the remaining 5 folds form the train set, with the process yield 6 test accuracy rates (Figure 12) and F1 scores (Figure 13) for each method.

4.1.3 Discussion and Conclusion

Performance Analysis and Overall Comparison Figures 12 and 13 demonstrate that the proposed model, when trained with the selected four-feature configuration, consistently achieves comparable or slightly superior performance relative to previously published linear and tree-based approaches across all evaluated datasets. Although the observed improvements in mean accuracy and F1 score are modest, they are stable across cross-validation folds, indicating that the gains are systematic rather than incidental.

In particular, increasing the number of features from one to two and then to four generally leads to improved predictive performance for all model families considered. This trend suggests that the selected features provide complementary information that enhances the model's ability to correctly identify changepoint intervals. However, the inclusion of all available features does not necessarily yield further improvement. In several cases, performance stagnates or slightly deteriorates when all features are used, indicating the presence of redundant or weakly informative variables. This highlights the importance of careful feature selection, especially for flexible models such as MLPs.

Comparison Between Model Families A notable observation is that tree-based models do not consistently outperform linear models when trained on the same feature sets. This phenomenon aligns with findings reported in [Drouin et al., 2017] for the Systematic dataset and in [Barnwal et al., 2022b] for subsets of the Epigenomic dataset. The primary reason appears to be the predominantly linear relationship between the extracted features and the target intervals, as illustrated in Figure 11. When the underlying structure of the data is largely linear, more complex nonlinear learners such as tree ensembles do not offer a significant advantage.

The proposed model, formulated as a piecewise linear MLP, is conceptually positioned between purely linear models and highly nonlinear deep architectures. As expected, for identical feature configurations, it generally outperforms the standard linear model. The improvement can be attributed to its ability to capture mild nonlinearities while preserving much of the interpretability and stability of linear approaches. Importantly, strong performance is achieved with a relatively

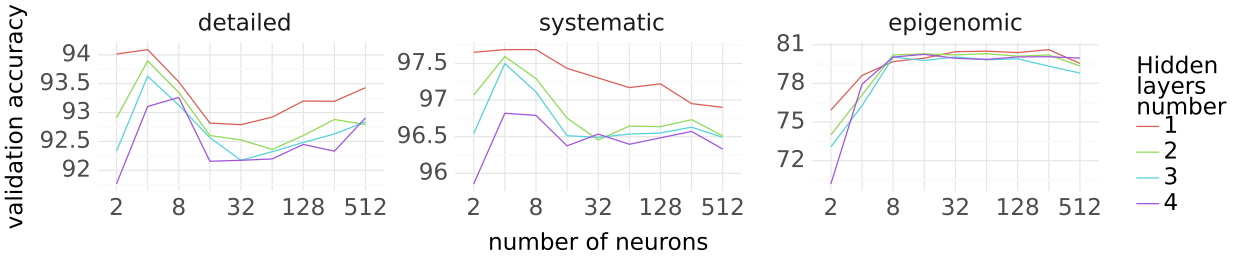

Figure 14: Average validation accuracies for different MLP configurations

simple architecture—typically a single hidden layer containing 4 or 8 neurons—demonstrating that excessive architectural complexity is unnecessary for these datasets.

Consistency and Stability of the Proposed Model While the proposed model achieves strong mean performance, Figures 12 and 13 also indicate that MLPs tend to exhibit wider confidence intervals compared to linear and tree-based models. This variability likely stems from the higher sensitivity of neural networks to initialization, optimization dynamics, and hyperparameter configurations. Despite this increased variance, the overall performance remains competitive and robust across folds. The results suggest that moderate architectural simplicity contributes to improved stability without sacrificing predictive power.

Factors Affecting Performance

- *Feature Design.* The experiments confirm that feature quality plays a more critical role than model complexity. The relationship between sequence length and the target interval of λ, as shown in Figure 11, is not clearly separable, which may explain why models trained solely on this feature underperform relative to those incorporating additional statistical descriptors such as variance or range. The fact that nonlinear models do not benefit from the inclusion of all features further underscores that feature relevance, rather than feature quantity, determines performance gains.

- *MLP Configuration.* Determining an effective MLP configuration required systematic exper-

imentation. We evaluated 36 distinct configurations for each train–test pair by varying the number of hidden layers (1–4) and the number of neurons per layer (2 to 512, in powers of two). This grid search was computationally demanding. Nevertheless, results indicate that shallow architectures with limited width are sufficient. Given the largely linear structure of the problem, deeper or wider networks did not provide consistent improvements, reinforcing the principle that model capacity should match data complexity.

Limitations Several limitations should be acknowledged. First, feature extraction was performed manually, and only a limited subset of possible statistical descriptors was considered. The space of potential features derivable from sequences is effectively unbounded; measures such as skewness, kurtosis, entropy, or counts of repeated values were not included. Consequently, potentially informative predictors may have been overlooked.

Second, unlike L1-regularized linear models or tree-based algorithms, MLPs lack an inherent feature selection mechanism. This limitation reduces flexibility when handling larger feature spaces and increases the risk of incorporating redundant variables. As a result, careful preprocessing remains essential for neural network models in this setting.

Third, identifying optimal MLP hyperparameters is time-consuming and computationally intensive. Although an extensive search was conducted, configurations with heterogeneous neuron counts across layers were not explored, leaving open the possibility that alternative architectures could yield marginal improvements.

Conclusion and Future Directions Overall, the proposed piecewise linear MLP model achieves comparable or slightly improved accuracy and F1 scores in changepoint detection relative to traditional linear and tree-based models. The results emphasize that thoughtful feature selection is more influential than architectural complexity. While the model benefits from moderate nonlinear flexibility, its performance does not justify highly complex network designs for the datasets studied.

Future research may focus on automating feature learning rather than relying on manual ex-

traction. In particular, sequence-oriented neural architectures such as Recurrent Neural Networks (RNNs) [Rumelhart et al., 1986], Gated Recurrent Units (GRUs) [Chung et al., 2014], and Long Short-Term Memory (LSTM) networks [Hochreiter, 1997] could be investigated for directly learning informative representations from raw sequences. Such approaches may alleviate the need for handcrafted features and potentially enhance generalization to other sequence domains.

Reproducible Research Material To promote transparency and facilitate replication, all source code and experimental materials are publicly available at: github. Providing open access to the implementation ensures that results can be independently verified and extended by future researchers.

4.2 Optimal Partitioning Penalty Learning: Recurrent

Changepoint detection (other words, partitioning or segmentation) is important for identifying abrupt changes in data in various domains, including finance [Lattanzi and Leonelli, 2021], healthcare [Muggeo and Adelfio, 2010], network security [Tartakovsky et al., 2013], and environmental science [Reeves et al., 2007]. This importance has driven the development of numerous methods over the past decades.

Changepoint Detection Methods. Changepoint detection methods can be divided into two categories: Deep Learning (DL) and Dynamic Programming (DP).

DL-based methods, which inspired from a statistical method Cumulative Sum (CUSUM) [Hinkley, 1971, James et al., 1987], including recent works [Li et al., 2024, Ermshaus et al., 2023] employ multilayer perceptrons (MLP) or k-nearest neighbors (KNN) as classification sliding windows to identify changepoints. However, these methods rely on local neighbors rather than the entire sequence and are overly complex for univariate data.

In contrast, DP is well-suited for this context. These algorithms rely on a single hyperparameter to limit the changepoints number, which can be either: (a) a fixed changepoints number or (b) a penalty parameter to penalize the changepoint presence. Fixed changepoints number allow DP algorithms [Auger and Lawrence, 1989b, Bai and Perron, 2003b, Killick et al., 2012b, Rigaill, 2015] to locate them, but the changepoints number is rarely predetermined. More commonly used, penalty-based DP algorithms, such as Optimal Partitioning (OPART) [Jackson et al., 2005b] and its variants—Pruned Exact Linear Time (PELT) [Killick et al., 2012b], Functional Pruning Optimal Partitioning (FPOP) [Maidstone et al., 2016b], and Labeled Optimal Partitioning (LOPART) [Hocking and Srivastava, 2023b]—are widely regarded as the most effective. PELT and FPOP prune candidate changepoints efficiently, producing identical partitions as OPART, while LOPART extends OPART by incorporating predefined labels (the expected number of changepoints within specific location ranges) or defaulting to OPART when labels are absent. The following is the OPART optimization problem:

OPART Problem. Given a sequence $\mathbf{d} \in \mathbb{R}^N$ and penalty parameter $\lambda \geq 0$, find the vector $\mathbf{m} \in \mathbb{R}^N$ that minimizes the following cost function:

$$C(\mathbf{d}, \mathbf{m}) + \lambda M$$

Here, $C(\mathbf{d}, \mathbf{m})$ represents the error between the sequence $\mathbf{d}$ and the vector $\mathbf{m}$ and M is the number of changes in $\mathbf{m}$. The formula of $C(\mathbf{d}, \mathbf{m})$ and M depends on the type of change. If the type of change is mean, then $C(\mathbf{d}, \mathbf{m}) = \sum_{i=1}^{N}(m_i - d_i)^2$ and $M = \sum_{i=1}^{N-1} I[m_i \neq m_{i+1}]$, where $I[m_i \neq m_{i+1}]$ equals 1 if a changepoint occurs ($m_i \neq m_{i+1}$) and 0 otherwise.

In OPART algorithms, the penalty plays a key role. A higher penalty imposes a stronger penalty on changepoint presence, leading to fewer changepoints, see Figure 7. This study aims to predict that penalty to improve the changepoint detection accuracy. From here on, that conceptual penalty will be referred to as λ.

Existing λ prediction models. Various models exist for predicting λ, including unsupervised [Schwarz, 1978, Akaike, 1974, Lavielle, 2005] and supervised such as linear [Rigaill et al., 2013b] and tree(s) [Drouin et al., 2017, Barnwal et al., 2022b]. But all of these models rely on a set of manually extracted sequence features, yet the number of possible features is infinite, this process can result in the inclusion of many irrelevant features, and the potential omission of useful ones.

(iv) (v) Contribution. This study proposes a novel method utilizing a recurrent network to automatically extract relevant features from sequences, which are then used to predict λ. Experiments on benchmark datasets show that the proposed method achieves higher partitioning accuracy than existing ones in most cases.

Reproducibility. To ensure transparency, the code used for this study is accessible at: `https://github.com/lamtung16/ML_Changepoint_Detection_epigenomic_rnn`.

4.2.1 Contribution

Feature Extraction via Recurrent Networks. In a Recurrent Network with input size $k \geq 1$ and hidden size $m \geq 1$, processing a sequence $\mathbf{d} = [d_1, d_2, \ldots, d_N] \in \mathbb{R}^N$, the hidden state at time t, called h_t, is updated as:

$$h_t = f(d_{[t:t+k-1]}; h_{t-1}) \in \mathbb{R}^m$$

where $d_{[t:t+k-1]} = [d_t, d_{t+1}, ..., d_{t+k-1}] \in \mathbb{R}^k$. Time stamp t ranges from k to N. The initial hidden state h_{k-1} can be assigned manually. The final hidden state $h_N \in \mathbb{R}^m$ captures information from the entire sequence. Thus, we use h_N as the m extracted features from the raw sequence.

Predicting λ from extracted features. The prediction is formed by the linear combination of m extracted features:

$$\log \lambda = h_N \cdot \beta + \beta_0$$

where $\beta \in \mathbb{R}^m$ and $\beta_0 \in \mathbb{R}$ are the parameters to be learned.

Automatic extracted features vs. Manual extracted features. The novel method generates useful features without the need for a large number, unlike previous models. Figure 15 provides an example by comparing the ability to predict λ using length (or variance) - as these manual extracted features are considered key for λ prediction in unsupervised methods - versus automatic extracted features from recurrent networks. It demonstrates that the automatic extracted features probably provide a more accurate prediction of λ than either length or variance.

Feature Approximation Ability of Recurrent Networks. Recurrent networks have the ability to approximate statistical sequence features. For example, if the core network is defined as $h_t = h_{t-1} + 1$ when $h_0 = 0$ and the input size $k = 1$, then $h_N = N$, which represents the sequence length, a feature used in BIC. The core component of these models is a deep neural network. For instance, the Vanilla Recurrent Network [Rumelhart et al., 1986] core is a MLP, which, according

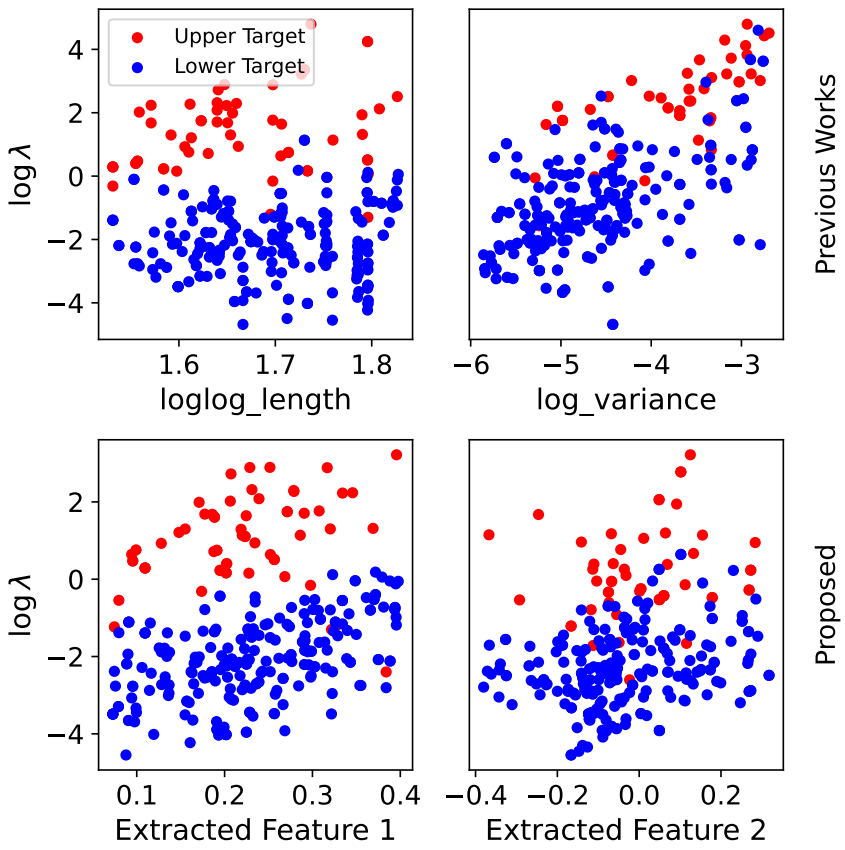

Figure 15: Example: features vs. targets from the dataset detailed

to the universal approximation theorem [Hornik et al., 1989a], can approximate any continuous function, regardless of its complexity, provided there are enough neurons in the hidden layer. This is why, although these models may not directly produce exact statistical sequence features, their architecture enables them to approximate these features with a high degree of accuracy.

Recurrent networks to be implemented. This study explores three recurrent networks: Vanilla Recurrent Network (RNN), Long Short-Term Memory (LSTM) [Hochreiter, 1997], Gated Recurrent Unit (GRU) [Chung et al., 2014]. RNNs struggle with long-term dependencies due to the vanishing gradient problem. LSTMs address this by introducing memory cells and gating mechanisms that help retain information over longer sequences. GRUs, a simplified version of LSTMs,

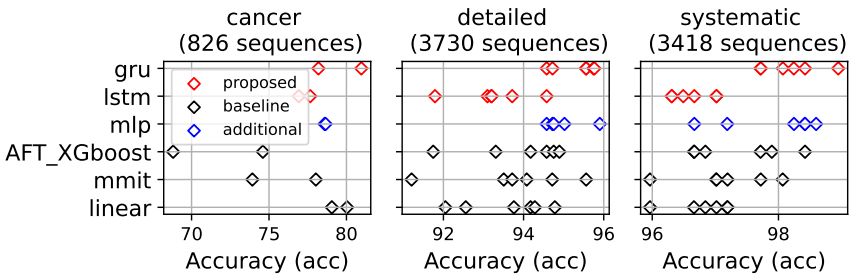

Figure 16: The accuracies for each fold are presented for datasets without any pooling

combine the forget and input gates into a single update gate, offering a more efficient alternative while still capturing long-term dependencies.

4.2.2 Experiments and Results

The comprehensive accuracy results of all 8 methods for each of 20 datasets are provided in the Appendix, as the table is too large to be included in the main paper. Figure 16 presents the accuracy rates for three datasets using the original, unpooled sequences. Each dataset is evaluated using seven different methods, producing multiple accuracy values depending on the number of folds used for that dataset.

4.2.3 Discussion and Conclusion

Summary of Contributions. Technically, these proposed models still perform two steps: (1) Feature extraction and (2) Predicting λ based on the extracted features. However, instead of performing these steps separately, it combines them into a single operation, enhancing its robustness compared to previous models. Automatic extracting features helps generate highly useful ones without requiring many features, as shown in Figure 15. This study focuses exclusively on genomic datasets, where prior knowledge of relevant features enables effective manual feature extraction. However, the proposed models stand out because it does not require expert knowledge to perform. Its automatic feature extraction mechanism makes it versatile and applicable to any type of sequence

dataset.

Comparison to Previous Models. Recurrent networks, in general, outperform previous models across datasets, see Figure 16. For small datasets (with fewer than 100 instances), the accuracy of the proposed models is comparable (neither better nor worse) to that of existing models. On the other hand, when dealing with larger datasets, recurrent networks noticeably outperform previous models and show a slight edge over MLPs.

Comparison between Recurrent Networks. When comparing RNN, LSTM, and GRU, as expected, RNN performs the worst due to its difficulty with long-term memory. In most cases, GRU outperforms the others.

Discussion on the Number of Automatically Extracted Features. The number of automatic extracted features needs to be sufficiently large. It is observed that when 8 or 16 features are extracted, the validation loss value is slightly smaller compared to when only 2 or 4 features are extracted.

Impact of pooling preprocessing. Shortening the sequence for training is effective. As the window size increases, more information is lost. However, when the window size is small, such as 100, the performance of the recurrent models is worse compared to larger window sizes like 1000 or 2000. This could be because the pooled sequence with a window size of 100 is still too long for the recurrent models to handle effectively. The pooling process is beneficial in two ways: it can improve the accuracy of recurrent networks and also reduce training time. We employed two pooling functions, mean and median, to represent a single value for each non-overlapping window. It is evident that using the mean value as the representation consistently yields smaller loss.

Study Limitations. Although the Recurrent Network architecture is straightforward, as its core essentially a deep neural network, training these models can be time-consuming due to the inher-

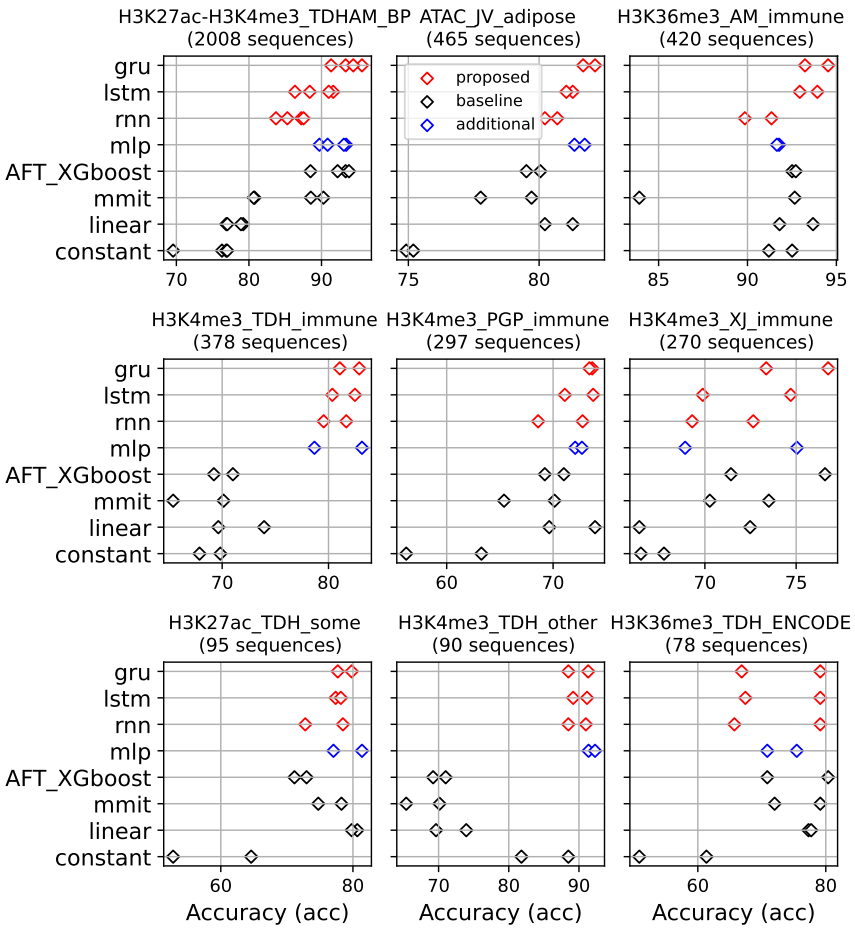

Figure 17: The fold accuracies on some ChIP-seq datasets

ently sequential nature of the process. For some train/test pairs, even after applying pooling to reduce the sequence length, training a model with a maximum of 2 layers, each containing no more than 16 neurons, and a maximum of 1000 iterations, took over approximately 10 days, despite utilizing the powerful NVIDIA A100 GPU. These models also require significant memory to store the hidden states at each time step. To avoid out-of-memory (OOM) errors, it is crucial to allocate sufficient memory, especially for long sequences.

Practical Considerations in Real-World Applications. There is a trade-off between training time and accuracy. The proposed models have higher accuracy in most of the 20 datasets, but they

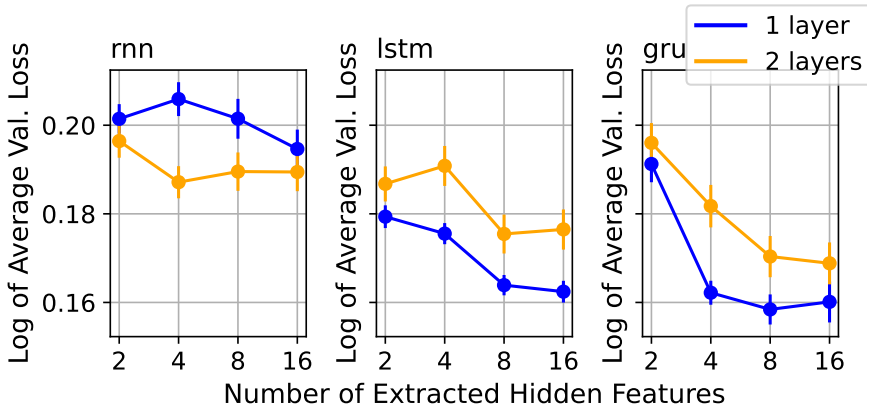

Figure 18: Log of the loss validation across different numbers of extracted hidden layers

are significantly more time-consuming to train, especially for longer sequences. In contrast, manually extracted features models do not face this issue. Therefore, in real-world scenarios where datasets contain only a few sequences but each sequence is long, and training time is a priority, manually extracted features models may be the better choice for two reasons: first, their training time is significantly faster; and second, these models' accuracy is comparable to the proposed models. However, if the user's priority is accuracy and the hardware requirements are met, the proposed models are a better choice. Although the training process is time-consuming, it only needs to be done once; afterwards, the trained models can be applied to unseen sequences, as their feed-forward process is sufficiently fast.

Future Directions. For these particular datasets, it is assumed that λ exhibits an increasingly monotonic behavior with respect to both sequence length and variance, which are employed in unsupervised methods. One approach is to leverage monotonic network architectures, such as min-max networks [Sill, 1997], or more advanced architectures like Lattice Networks [You et al., 2017], with modifications to the loss function for each linear region.

We can consider CNNs [LeCun et al., 1998] because they are capable of handling sequences of varying lengths and generating a single output by utilizing convolution layers to extract features

across the sequence. Global pooling layers (e.g., Global Max or Average Pooling) aggregate these features into a fixed-size representation. This vector is then passed through a MLP to produce the final output.

To build upon the initial concept of utilizing recurrent networks in this study, various types of models could be explored in the future:

- Using a more complex model: This study predicts λ by using a linear combination of the extracted features. Alternatively, we could consider using a more complex model such as MLP that processes the extracted features instead of relying on a linear combination.

- Attention mechanisms [Bahdanau et al., 2014] dynamically focus on relevant input parts, assigning importance weights to subsequences. Unlike sequential RNNs, they effectively capture long-range dependencies and handle varying input lengths, making them ideal for this task.

- Bidirectional Recurrent Networks [Schuster and Paliwal, 1997]: The models implemented in this study are unidirectional, leading to challenges with long-term memory, since the later parts of the sequence tend to be more important than the earlier ones. Using a bidirectional recurrent network architecture can aim to address this limitation, enabling the model to better capture information from the sequence.

Conducting even a preliminary experiment would require substantial computing resources, since bidirectional networks and attention mechanisms are more computationally expensive than the proposed one. For this reason, we chose not to include such experiments as part of this work.

4.3 Interval Regression: Review

In addition to existing state-of-the-art models, this study proposes several alternative models, including common machine learning architectures such as KNN, Neural Networks, and Random Forests, to provide a more comprehensive comparison.

4.3.1 Contribution

K-Nearest Neighbors (KNN) KNN is a classical model for Standard Regression, which motivates its consideration in the Interval Regression setting. The Interval Regression procedure follows the same steps as the standard KNN regression model [Fix and Hodges, 1985], with the key difference being how the regression value is determined from the k nearest neighbors. We propose utilizing the MMIT regression function. Specifically, we treat the set of k nearest neighbors as a single leaf in a MMIT, where the regression value is a constant that minimizes the mean Hinge Error (4) between this constant and the k target intervals. The prediction $\hat{y}$ based on the target intervals of the k nearest neighbors, denoted as $\{(y_l^i, y_u^i)\}_{i=1}^k$, is formulated as:

$$\hat{y} = \underset{c}{\operatorname{argmin}} \frac{1}{k} \sum_{i=1}^{k} l\big(c, (y_l^i, y_u^i)\big), \quad c \in \{y_l^i, y_u^i, \frac{y_l^i + y_u^i}{2} | y_l^i > -\infty, y_u^i < \infty\}_{i=1}^k$$

Multilayer Perceptron (MLP) Since MLP is a commonly used neural network in Standard Regression models [Hornik et al., 1989b], its application in Interval Regression is worth considering. The training process of this model is the same as in the linear model *Max Margin Interval Regression*.

Maximum Margin Interval Forest (MMIF) The idea of using ensembles of MMITs was mentioned in Chapter 6 (Discussion and Conclusions) of [Drouin et al., 2017]. Since MMIT is a tree for the Interval Regression setting, its extension to construct a Random Forest for Interval Regression is straightforward. The operation of MMIF follows the same principles as the standard Random Forest regressor [Breiman, 2001], with the key distinction being that MMITs are used instead of

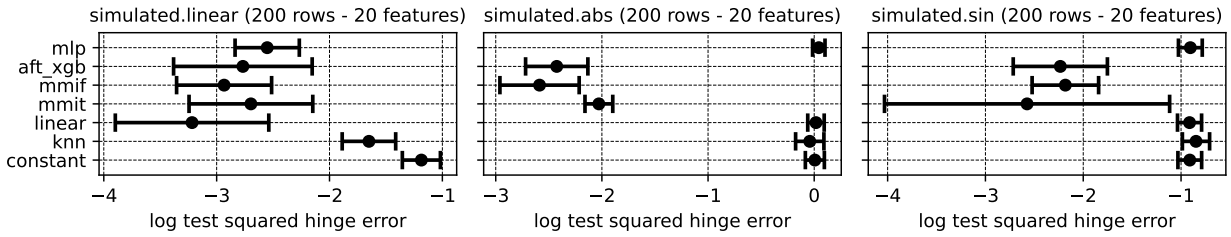

Figure 19: The mean and standard deviation of the test squared hinge errors in simulated datasets

conventional decision trees.

4.3.2 Experiments and Results

For each of the 39 datasets, 7 models were implemented, with each model producing 5 hinge error values (corresponding to 5 train/test pairs).

4.3.3 Discussions and Conclusions

Dataset Variations. To ensure a comprehensive comparison, the quality of the datasets is important. The datasets employed have a variety of characteristics, including high- and low-dimensional data, real-world and synthetic sources, linear and nonlinear patterns, as well as low- and high-noise conditions.

Comprehensive Comparison. The performance and consistency rankings are in Table 6, 1st means the best and 7th means the worst. About the performance, it's clear that MMIF outperforms other models in most cases. Linear, MMIT and MLP are comparable, while Constant, kNN, and the AFT model in XGBoost show the poorest performance. About the consistency, MMIF stands out as the most consistent. This is expected, as MMIF being an ensemble model, relies on a group of MMITs to make the final prediction, which helps improve its stability. Again, Constant, kNN, and the AFT model in XGBoost show the lowest consistency.

No single model is optimal for all scenarios. As shown in Table 6, there is no universally optimal model for every scenario. However, if complexity is not a primary concern, and the goal is to select the model with the best overall performance and consistency, MMIF is likely the first choice. Alternatively, if model complexity and training time are critical factors, MMIT is preferable, as it's less complex than MMIF, require significantly fewer training resources, and still maintain reliable performance.

MMIT vs MMIF. This is a comparison worth considering. MMIF was proposed with the expectation that an ensemble of MMITs would outperform a single MMIT, and the results confirm this hypothesis. Specifically, across 39 datasets, MMIF outperformed MMIT in 28 of them. This makes sense: MMIF, being an ensemble of MMITs, is able to reduce overfitting by not relying on a single MMIT.

Discussion on the AFT Model in XGBoost. The AFT model in XGBoost is the complex models in this study, but its performance and consistency do not justify the complexity. To make the model applicable to a general Interval Regression setting, the target intervals must be transformed using exponential function. So the model becomes highly sensitive when the dataset contains many left-censored target intervals. When predictions are made on the exponential scale, if the predicted value is close to zero, the mapping back to the original scale can result in large negative values.

$$(-\infty, y_u) \xrightarrow{\text{exp}} (0, \exp y_u) \xrightarrow{\text{predict}} y_p \approx 0 \xrightarrow{\log} \hat{y} \approx -\infty \xrightarrow{\text{evaluate}} \text{Big test error}$$

The Figure 2.b in [Barnwal et al., 2022a] indicates that the AFT model in XGBoost performs among the worst across their six simulated datasets when compared to the Linear and MMIT.

A potential direction for future work is to transform left-censored intervals into interval-censored intervals by replacing all lower bounds of $-\infty$ with a finite value. This adjustment aims to reduce the model's sensitivity to left-censored intervals and mitigate the tendency to produce excessively large negative predictions.

	Performance							Consistency						
	1st	2nd	3rd	4th	5th	6th	7th	1st	2nd	3rd	4th	5th	6th	7th
constant	2	2	3	1	4	7	20	4	0	2	3	3	8	19
knn	4	4	2	6	6	16	1	4	3	3	7	7	12	3
linear	3	11	6	8	5	5	1	7	7	8	7	7	3	0
mmit	2	10	9	3	9	4	2	2	13	4	8	8	2	2
mmif	15	5	8	7	4	0	0	12	9	9	4	3	2	0
aft_xgb	4	4	3	4	7	5	12	4	1	7	2	3	10	12
mlp	9	3	8	10	4	2	3	6	6	6	8	8	2	3

Table 6: The comparison of the performance and consistency of 7 interval regression models

References

O. Abbara and M. Zevallos. Modeling and forecasting intraday var of an exchange rate portfolio. *Journal of Forecasting*, 37(7):729–738, Aug. 2018. ISSN 1099-131X. doi: 10.1002/for.2540. URL `http://dx.doi.org/10.1002/for.2540`.

H. Akaike. A new look at the statistical model identification. *IEEE transactions on automatic control*, 19(6):716–723, 1974.

I. E. Auger and C. E. Lawrence. Algorithms for the optimal identification of segment neighborhoods. *Bulletin of mathematical biology*, 51(1):39–54, 1989a.

I. E. Auger and C. E. Lawrence. Algorithms for the optimal identification of segment neighborhoods. *Bulletin of mathematical biology*, 51(1):39–54, 1989b.

D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014. URL `https://api.semanticscholar.org/CorpusID:11212020`.

J. Bai and P. Perron. Computation and analysis of multiple structural change models. *Journal of applied econometrics*, 18(1):1–22, 2003a.

J. Bai and P. Perron. Computation and analysis of multiple structural change models. *Journal of applied econometrics*, 18(1):1–22, 2003b.

A. Barnwal, H. Cho, and T. Hocking. Survival regression with accelerated failure time model in xgboost. *Journal of Computational and Graphical Statistics*, 31(4):1292–1302, 2022a.

A. Barnwal, H. Cho, and T. Hocking. Survival regression with accelerated failure time model in xgboost. *Journal of Computational and Graphical Statistics*, 31(4):1292–1302, 2022b.

A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM journal on imaging sciences*, 2(1):183–202, 2009.

H. Binder and M. Schumacher. Allowing for mandatory covariates in boosting estimation of sparse high-dimensional survival models. *BMC bioinformatics*, 9:1–10, 2008.

L. Breiman. Random forests. *Machine learning*, 45:5–32, 2001.

L. Breiman. *Classification and regression trees*. Routledge, 2017.

L. Breiman, J. H. Friedman, R. A. Olshen, and C. Stone. Classification and regression trees (cart). *Biometrics*, 40(3):358, 1984.

T. Cai, J. Huang, and L. Tian. Regularized estimation for the accelerated failure time model. *Biometrics*, 65(2):394–404, 2009.

T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *KDD*, 2016.

X. Cheng, Y. Cao, X. Li, B. An, and L. Feng. Weakly supervised regression with interval targets. In *International Conference on Machine Learning*, pages 5428–5448. PMLR, 2023.

J. Chung, C. Gulcehre, K. Cho, and Y. Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.

Drouin. *Maximum Margin Interval Trees*, 2017. URL https://github.com/aldro61/mmit.

A. Drouin, T. D. Hocking, and F. Laviolette. Maximum margin interval trees. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 4954–4963, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

D. Ellison, M. Lundblad, and H. Petersson. Reforming the eu approach to lulucf and the climate policy framework. *Environmental Science and amp; Policy*, 40:1–15, June 2014. ISSN 1462-9011. doi: 10.1016/j.envsci.2014.03.004. URL http://dx.doi.org/10.1016/j.envsci.2014.03.004.

A. Ermshaus, P. Schäfer, and U. Leser. Clasp: parameter-free time series segmentation. *Data Mining and Knowledge Discovery*, 37(3):1262–1300, 2023.

M. Fazil Baksh, G. Haars, S. Todd, P. A. H. Van Noord, and J. Whitehead. Comparing correlations of continuous observations from two independent populations using a sequential approach. *Statistics in Medicine*, 25(24):4293–4310, Aug. 2006. ISSN 1097-0258. doi: 10.1002/sim.2676. URL http://dx.doi.org/10.1002/sim.2676.

E. Fix and J. L. Hodges. *Discriminatory Analysis: Nonparametric Discrimination and Consistency Properties*, volume 1. USAF School of Aviation Medicine, Randolph Air Force Base, TX, 1985.

J. H. Friedman. Multivariate adaptive regression splines. *The annals of statistics*, 19(1):1–67, 1991.

J. H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. ISSN 00905364, 21688966. URL http://www.jstor.org/stable/2699986.

M. G. Hamed, M. Serrurier, and N. Durand. Possibilistic knn regression using tolerance intervals. In S. Greco, B. Bouchon-Meunier, G. Coletti, M. Fedrizzi, B. Matarazzo, and R. R. Yager, edi-

tors, *Advances in Computational Intelligence*, pages 410–419, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. ISBN 978-3-642-31718-7.

T. J. Hastie. Generalized additive models. In *Statistical models in S*, pages 249–307. Routledge, 2017.

K. Hawkins. Survival analysis: Techniques for censored and truncated data klein jp, moeschberger ml (2003) isbn 038795399x; 536 pages;£ 69.00; 89.95; 94.00 springer-verlag; http://www. springer-ny. com/detail. tpl? isbn= 038795399x, 2004.

D. V. Hinkley. Inference about the change-point from cumulative sum tests. *Biometrika*, 58(3): 509–523, 1971.

S. Hochreiter. Long short-term memory. *Neural Computation MIT-Press*, 1997.

T. D. Hocking. *penaltyLearning: Penalty Learning*, 2024. URL `https://github.com/tdhock/penaltylearning`. R package version 2024.1.25.

T. D. Hocking and A. Srivastava. Labeled optimal partitioning. *Computational Statistics*, 38(1): 461–480, 2023a. ISSN 0943-4062. doi: 10.1007/s00180-022-01238-z.

T. D. Hocking and A. Srivastava. Labeled optimal partitioning. *Computational Statistics*, 38(1): 461–480, 2023b. ISSN 0943-4062. doi: 10.1007/s00180-022-01238-z.

K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989a.

K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989b.

T. Hothorn, B. Lausen, A. Benner, and M. Radespiel-Tröger. Bagging survival trees. *Statistics in medicine*, 23(1):77–91, 2004.

J. Huang, S. Ma, and H. Xie. Regularized estimation in the accelerated failure time model with high-dimensional covariates. *Biometrics*, 62(3):813–820, 2006a.

J. Huang, S. Ma, and H. Xie. Regularized estimation in the accelerated failure time model with high-dimensional covariates. *Biometrics*, 62(3):813–820, 2006b.

H. Ishwaran, U. B. Kogalur, E. H. Blackstone, and M. S. Lauer. Random survival forests. *BMC bioinformatics*, 2008.

B. Jackson, J. D. Scargle, D. Barnes, S. Arabhi, A. Alt, P. Gioumousis, E. Gwin, P. Sangtrakulcharoen, L. Tan, and T. T. Tsai. An algorithm for optimal partitioning of data on an interval. *IEEE Signal Processing Letters*, 12(2):105–108, 2005a.

B. Jackson, J. D. Scargle, D. Barnes, S. Arabhi, A. Alt, P. Gioumousis, E. Gwin, P. Sangtrakulcharoen, L. Tan, and T. T. Tsai. An algorithm for optimal partitioning of data on an interval. *IEEE Signal Processing Letters*, 12(2):105–108, 2005b.

B. James, K. L. James, and D. Siegmund. Tests for a change-point. *Biometrika*, 74(1):71–83, 1987.

J.-T. Jeng, C.-C. Chuang, and S.-F. Su. Support vector interval regression networks for interval regression analysis. *Fuzzy Sets and Systems*, 138(2):283–300, 2003. ISSN 0165-0114. doi: https://doi.org/10.1016/S0165-0114(02)00570-5. URL https://www.sciencedirect.com/science/article/pii/S0165011402005705.

R. Killick, P. Fearnhead, and I. A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012a.

R. Killick, P. Fearnhead, and I. A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012b.

C. Lattanzi and M. Leonelli. A change-point approach for the identification of financial extreme regimes. *Brazilian Journal of Probability and Statistics*, 35(4), Nov. 2021. ISSN 0103-0752. doi: 10.1214/21-bjps509. URL http://dx.doi.org/10.1214/21-BJPS509.

M. Lavielle. Using penalized contrasts for the change-point problem. *Signal Processing*, 85(8): 1501–1510, 2005. ISSN 0165-1684. doi: https://doi.org/10.1016/j.sigpro.2005.01.012.

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

J. Lee, Y. Xie, and X. Cheng. Training neural networks for sequential change-point detection. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023.

A.-M. Legendre. *Nouvelles méthodes pour la détermination des orbites des comètes, avec un supplément contenant divers perfectionnements de ces méthodes et leur application aux deux comètes de 1805*. book, 1805. URL `https://books.google.com/books?id=5nJUAAAAcAAJ`. Courcier.

G. Levitin, L. Xing, H. Ben-Haim, and Y. Dai. Multi-state systems with selective propagated failures and imperfect individual and group protections. *Reliability Engineering amp; System Safety*, 96(12):1657–1666, Dec. 2011. ISSN 0951-8320. doi: 10.1016/j.ress.2011.08.002. URL `http://dx.doi.org/10.1016/j.ress.2011.08.002`.

J. Li, P. Fearnhead, P. Fryzlewicz, and T. Wang. Automatic change-point detection in time series via deep learning. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86 (2):273–285, 2024.

M. Londschien, P. Bühlmann, and S. Kovács. Random forests for change point detection. *Journal of Machine Learning Research*, 24(216):1–45, 2023.

S. M. Lundberg, G. G. Erion, and S.-I. Lee. Consistent individualized feature attribution for tree ensembles. *arXiv preprint arXiv:1802.03888*, 2018.

R. Maidstone, T. Hocking, G. Rigaill, and P. Fearnhead. On optimal multiple changepoint algorithms for large data. *Statistics and Computing*, 27(2):519–533, Feb. 2016a. ISSN

1573-1375. doi: 10.1007/s11222-016-9636-3. URL http://dx.doi.org/10.1007/s11222-016-9636-3.

R. Maidstone, T. Hocking, G. Rigaill, and P. Fearnhead. On optimal multiple changepoint algorithms for large data. *Statistics and Computing*, 27(2):519–533, Feb. 2016b. ISSN 1573-1375. doi: 10.1007/s11222-016-9636-3. URL http://dx.doi.org/10.1007/s11222-016-9636-3.

G. F. Montufar, R. Pascanu, K. Cho, and Y. Bengio. On the number of linear regions of deep neural networks. *Advances in neural information processing systems*, 27, 2014.

V. M. R. Muggeo and G. Adelfio. Efficient change point detection for genomic sequences of continuous measurements. *Bioinformatics*, 27(2):161–166, 11 2010. ISSN 1367-4803. doi: 10.1093/bioinformatics/btq647. URL https://doi.org/10.1093/bioinformatics/btq647.

S. Pölsterl. scikit-survival: A library for time-to-event analysis built on top of scikit-learn. *Journal of Machine Learning Research*, 21(212):1–6, 2020.

S. Pölsterl, N. Navab, and A. Katouzian. An efficient training algorithm for kernel survival support vector machines. *arXiv preprint arXiv:1611.07054*, 2016.

J. Reeves, J. Chen, X. L. Wang, R. Lund, and Q. Q. Lu. A review and comparison of changepoint detection techniques for climate data. *Journal of Applied Meteorology and Climatology*, 46(6): 900–915, June 2007. ISSN 1558-8424. doi: 10.1175/jam2493.1. URL http://dx.doi.org/10.1175/JAM2493.1.

G. Rigaill. A pruned dynamic programming algorithm to recover the best segmentations with 1 to $k_\{max\}$ change-points. *Journal de la Société Française de Statistique*, 156(4):180–205, 2015.

G. Rigaill, T. D. Hocking, F. Bach, and J.-P. Vert. Learning Sparse Penalties for Change-Point

Detection using Max Margin Interval Regression, June 2013a. URL https://inria.hal.science/hal-00824075.

G. Rigaill, T. D. Hocking, F. Bach, and J.-P. Vert. Learning Sparse Penalties for Change-Point Detection using Max Margin Interval Regression, June 2013b. URL https://inria.hal.science/hal-00824075.

D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

M. Schuster and K. K. Paliwal. Bidirectional recurrent neural networks. *IEEE transactions on Signal Processing*, 45(11):2673–2681, 1997.

G. Schwarz. Estimating the Dimension of a Model. *The Annals of Statistics*, 6(2):461 – 464, 1978. doi: 10.1214/aos/1176344136. URL https://doi.org/10.1214/aos/1176344136.

J. Sill. Monotonic networks. *Advances in neural information processing systems*, 10, 1997.

H. Tanaka and H. Lee. Interval regression analysis by quadratic programming approach. *IEEE Transactions on Fuzzy Systems*, 6(4):473–481, 1998. doi: 10.1109/91.728436.

A. G. Tartakovsky, A. S. Polunchenko, and G. Sokolov. Efficient computer network anomaly detection by changepoint detection methods. *IEEE Journal of Selected Topics in Signal Processing*, 7 (1):4–11, 2013. doi: 10.1109/JSTSP.2012.2233713.

C. Truong, L. Gudre, and N. Vayatis. Penalty learning for changepoint detection. In *2017 25th European Signal Processing Conference (EUSIPCO)*, pages 1569–1573, 2017. doi: 10.23919/EUSIPCO.2017.8081473.

X. Wang, S. Li, and T. Denœux. Interval-valued linear model. *International Journal of Computational Intelligence Systems*, 8(1):114, 2015. ISSN 1875-6883. doi: 10.2991/ijcis.2015.8.1.10. URL http://dx.doi.org/10.2991/ijcis.2015.8.1.10.

L.-J. Wei. The accelerated failure time model: a useful alternative to the cox regression model in survival analysis. *Statistics in medicine*, 11(14-15):1871–1879, 1992a.

L.-J. Wei. The accelerated failure time model: a useful alternative to the cox regression model in survival analysis. *Statistics in medicine*, 11(14-15):1871–1879, 1992b.

S. You, D. Ding, K. Canini, J. Pfeifer, and M. Gupta. Deep lattice networks and partial monotonic functions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.